QSC Python Wrapper – QSCPY

Technical Specification · Native QSCABI 0.10 Interface · November 9, 2026

Quantum Resistant Cryptographic Solutions Corporation

This engineering specification documents the QSCPY Python interface, ctypes/native binding architecture, cryptographic and TLS services, owned handles, source-level API contracts, examples, qualification harness and deployment requirements. The public API inventory and native registry are extracted from the uploaded project archive.

QSCPY is part of the QRCS wrapper project. Applicable project license policy: QRCS Commercial Application Software License (QRCS-CASL). The uploaded ZIP does not contain a standalone license file; distributors must include the repository-authorized license text.

Contents

1 Foreword 3
2 Introduction and Source Baseline 3
3 References, Definitions and Conventions 4
4 Architecture and Native ABI Integration 6
5 Installation, Runtime Loading and Deployment 7
6 Buffers, Types, Status and Ownership 9
7 Version, Metadata and ABI Qualification 10
8 Encoding and Secure Memory 11
9 Hash, XOF and Streaming Digests 11
10 MAC and Key Derivation 12
11 Symmetric Encryption and Authenticated Decryption 12
12 Entropy and Random Services 13
13 Post-Quantum KEMs and Signatures 13
14 Classical Public-Key Operations 14
15 Explicit Algorithm Selection 14
16 Native Streaming State and Lifecycle 15
17 X.509 Certificates, Chains and Trust Stores 16

18 CSR Issuance, DER Signing and CRLs 16
19 TLS 1.3 Context, Connection and Server APIs 17
20 Hardware Provider Bridge 18
21 Python Integration Examples 18
22 Test Harness and Qualification Evidence 19
23 Security, Compatibility and Release Requirements 21
24 Complete Public Python API Reference 22
25 Native ctypes ABI Function Registry 72
Annex A Public Classes, Enums and Data Contracts 85
Annex B Source, Test and Example Inventory 87
Annex C Implementation and Deployment Checklist 88

1: Foreword

QSCPY is the Python-language wrapper for QSC cryptographic services. Public Python modules translate natural Python byte arrays, strings, option objects and exceptions into a configured native QSCABI interface; QSCABI subsequently calls the external QSC library. QSCPY does not implement the underlying ciphers or public-key constructions in pure Python.

This specification is designed for engineers maintaining the package and consumers of its API. It covers exported Python functions and properties, opaque native handles, ctypes struct layout, status translation, correct use of mutable secret buffers, language-level errors, and qualification boundaries. All declarations in the API reference come from the uploaded QSCPY(5).zip tree, not inferred from previously released wrappers.

The reference follows the QRCS specification format established for QSCABI, QSCRS and QSCCPP: US-letter pages, 1-inch margins, Segoe UI text, Segoe UI Semibold headings with no bold font runs, numbered sections, full-text-width tables, and black text except external hyperlinks. Its purpose is implementor accuracy rather than marketing claims.

2: Introduction and Source Baseline

2.1 Distribution contents

The uploaded package contains qsc/ with Python API modules, qsc/_native/ with ctypes types and function-binding declarations, examples/ with a console dispatcher, tests/ with an automated qualification runner, and tools/ with native-loader diagnostics and an ABI contract checker. A PE32+ x64 QSCABI.dll is present under native/, alongside a Visual Studio Python project and Windows helper scripts. The archive has no root pyproject.toml, setup.py or wheel metadata, so it should be treated as a source/IDE distribution, not a published pip-installable package.

2.2 Scope and evidence baseline

Property	Source-grounded value / limitation
Source snapshot	QSCPY(5).zip, uploaded November 9, 2026
Public Python entries	462 named module-level functions, class methods and properties; 19 are @property descriptors
Public classes	29 non-internal classes in 24 qsc/ API modules, including dataclasses and IntEnum

Property	Source-grounded value / limitation
Native symbols	329 registered qscabi_* function specs; exact agreement with frozen EXPECTED_EXPORTS
Native artifact	native/QSCABI.dll (PE x64, 2,180,608 bytes) included; not executed or ABI-tested for this paper
Testing	148 declared test methods across 22 test modules; declared tests are not recorded runtime passes
Examples	15 named example groups in examples/manifest.py; opt-in execution
Build and license	Visual Studio .pyproj and helpers present; no setup/pyproject and no repository-authorized license file in ZIP
Compatibility	Explicit native QSCABI 0.10 paths; older native functions may load but strict qualifier demands 329 exports

Table 2.1 QSCPY supplied-source baseline

2.3 Source precedence

The concrete Python function body, its module-local type conversions, the declarations in qsc/_native/bindings.py and the frozen QSCABI symbol list control this document. Generic automatically generated docstrings are not proof of runtime behavior. Test case names define intended checks but not a passing native result. Optional API availability remains a runtime property of the library actually loaded.

3: References, Definitions and Conventions

3.1 Primary project source files

- qsc/__init__.py — Public package exports and root namespace.
- qsc/_native/abi.py — Cross-platform lazy library discovery, DLL paths, caching and diagnostics.
- qsc/_native/bindings.py — ctypes function declarations, return and argument types for all 329 native C exports.
- qsc/_native/types.py — ctypes scalar types, enums, TLS/X.509 fixed-layout structures and callback prototypes.
- qsc/_native/handles.py — Opaque handle ownership, closed-state and deterministic context-manager support.
- qsc/_helpers.py and qsc/_abi10_helpers.py — Argument conversion, validated capacities, temporary output and status helpers.
- qsc/abi010.py — Frozen native export manifest, capability bits and strict ABI 0.10 qualification.

- qsc/tls.py, qsc/x509.py, qsc/issuance.py — Network and PKI object lifecycles, certificate issuance and callback signing.
- qsc/stream.py and explicit_*.py — Optional ABI 0.10 streaming and independent algorithm-selection APIs.
- tests/run_tests.py and tests/manifest.py — Sequential qualification console and canonical 22-section test catalog.
- examples/manifest.py and examples/dispatcher.py — 15 opt-in runnable example groups and CLI behavior.
- tools/abi010_frozen_bindings.json — 301 prior + 28 added native binding specification records.
- QSCPY.pyproj and run_*.cmd — Windows project/console integration and CLI entry points.

3.2 External implementor references

Python standard-library ctypes — https://docs.python.org/3/library/ctypes.html

Python unittest — https://docs.python.org/3/library/unittest.html

Python context manager protocol — https://docs.python.org/3/reference/datamodel.html#context-managers

Python buffer protocol — https://docs.python.org/3/c-api/buffer.html

RFC 8446 TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446

RFC 5280 X.509 PKI — https://www.rfc-editor.org/rfc/rfc5280

NIST FIPS 203 ML-KEM — https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204 ML-DSA — https://doi.org/10.6028/NIST.FIPS.204

NIST FIPS 205 SLH-DSA — https://doi.org/10.6028/NIST.FIPS.205

3.3 Terminology and notation

Term	Meaning
QSC	Native cryptographic library used indirectly by QSCPY through QSCABI.
QSCABI	Platform-native C ABI boundary; source of the 329 function symbols.
ctypes	Python foreign-function interface defining C scalar, pointer and structure types.
Safe high-level function	A Python call that validates input and returns bytes, dataclasses, booleans or raises QscError.
try_* operation	Status-returning Python entry point; caller must check status and byte count.
Owned handle	Native object whose lifecycle must be terminated with the matching destroy/close operation.
Borrowed pointer	Native/Python memory valid only for the documented call or owner lifetime.

Term	Meaning
Output capacity	Caller-supplied writable byte count; distinct from actual bytes produced.
ABI strict mode	Version and full-export preflight; does not establish security certification.
CPU capability report	OS-visible host features, not a declaration of QSC build ISA or runtime dispatcher.

Table 3.1 Implementor terminology

4: Architecture and Native ABI Integration

4.1 Three-layer call model

```
Application code / tests / examples (Python 3)
    | typed qsc.* functions and owned objects
    v
qsc public wrapper + qsc._native ctypes registry
    | ctypes.CDLL; exact 329 qscabi_* C symbols
    v
QSCABI native shared library (Windows DLL / Linux SO / macOS dylib)
    | QSC external native cryptographic and protocol implementations
    v
Native QSC library
```

The Python package performs input validation, capacity calculation, construction of ctypes buffers and status conversion; QSCABI owns any opaque native state. The public modules do not expose QSC source header layouts or inline native QSC structures. Integration is source-level Python and C-ABI binary dependent, not an independent cryptographic implementation.

4.2 Native symbol registry and FFI types

qsc/_native/bindings.py records AbiFunctionSpec(name, module, restype, argtypes) for 329 functions. The loader configures ctypes restype and argtypes on demand. c_size_t must follow host pointer width; c_uint32, c_uint64 and c_int retain native C widths. BytePtr is writable output memory, ConstBytePtr is borrowed input, Handle is c_void_p and SizeTPtr is a pointer to c_size_t. Misdeclaring one argument can corrupt memory or cause a crash even when Python import succeeds.

4.3 Frozen ABI contract

The frozen QSCABI 0.10 symbol list in qsc/abi010.py contains 329 names; all 329 appear exactly in the binding registry. Tools/abi010_frozen_bindings.json contains 301 prior

declaration records and 28 additional records. That static match validates source consistency, not the actual exports of the bundled DLL. ABI compatibility, loaded binary architecture and actual exported symbols must be checked at deployment.

5: Installation, Runtime Loading and Deployment

5.1 Source-root execution

```
cd QSCPY
python -B tests/run_tests.py --strict-abi --no-pause
python -B examples/__main__.py -list
python -B examples/__main__.py -run hash
python -B tools/check_native.py
```

These are project entry points and usage examples, not commands executed while preparing this specification. The Windows helpers run_tests.cmd, run_examples.cmd and run_version.cmd execute their corresponding Python scripts. The Visual Studio .pyproj supplies an IDE-oriented launch configuration. Because no pyproject.toml or setup.py is present in this ZIP, a standard pip install from this archive is not specified.

5.2 Loader search precedence

QscAbi.load maintains one process-wide library handle guarded by an RLock. The QSCABI_DLL environment variable supplies an absolute full path override. QSCABI_NATIVE_DIR supplies a directory; an explicit QscAbi.set_native_library_directory call also influences candidate order. The loader searches bundled native and conventional adjacent QSCABI build locations, then attempts platform library names through the system loader. Details and failed candidates appear in QscAbi.diagnostics().format().

QscAbi.reset clears the Python cached loader handle for diagnostics and tests; it does not guarantee the operating system has unloaded all native dependencies or safely invalidated live native handles. Set loader options before first native call and avoid changing the active native library while TLS, X.509, streaming or hardware objects exist.

5.3 Platform and binary compatibility

The supplied native/QSCABI.dll is a Windows x64 PE binary. On Linux the loader expects a platform-compatible shared object; on macOS a matching dynamic library. The ZIP

does not include Linux or macOS native libraries. A Python interpreter, QSCABI DLL/so/dylib, and linked QSC dependency must agree on architecture, loader runtime, QSC version, native struct layout, CPU instruction requirements and library permissions.

If the DLL fails to load, inspect QscAbi.diagnostics() and dependency loading before changing any cryptographic code. A missing downstream QSC dependency, wrong Visual Studio runtime, or incorrect x64/x86 pairing can produce an OS loader error that looks like a Python wrapper failure. An imported `qsc` package does not prove that the native ABI can resolve.

5.4 Release packaging

Deliver the package source with the correct native binary, dependency chain, tested search paths, supported Python versions and authorized QRCS-CASL license material. Do not treat the bundled Debug/Release DLL filename as a verified production build. Verify SHA-256 hashes, signing, native export list, architecture, and expected version through a repeatable release process.

6: Buffers, Types, Status and Ownership

6.1 Python input and result types

Most cryptographic entry points accept bytes | bytearray | memoryview and return bytes or structured dataclasses on success. Mutable bytearray is used for some secret-bearing outputs; however ordinary immutable Python bytes cannot be reliably erased after construction. Applications requiring stronger secret-lifetime controls must own mutable buffers and avoid unnecessary string/bytes copies.

6.2 Status conversion and failure policy

qsc.status.QscStatus defines integer values 0 through 18. `ensure_success` raises QscError for a non-success status; `try_*` methods return a status and frequently an explicit output size/result. A wrong signature, failed authentication, invalid size, unsupported implementation, closed handle, TLS failure and X.509 failure must remain distinguishable. Never use partially produced data after an unsuccessful native call.

Status	Numeric	Policy implication
SUCCESS	0	Only success permits use of output when all length checks pass.

Status	Numeric	Policy implication
INVALID_ARGUMENT / INVALID_SIZE	2 / 3	Reject unsuitable arguments before consuming result.
BUFFER_TOO_SMALL	4	Allocate correct output and retry only when the operation is documented as retry-safe.
AUTHENTICATION_FAILURE / VERIFICATION_FAILURE	6 / 7	Reject protected data/signature; never treat as a warning.
PLATFORM_UNAVAILABLE / UNSUPPORTED	8 / 9	Stop or select an explicitly authorized alternative; do not silently downgrade.
INVALID_HANDLE / BAD_STATE / CLOSED	12 / 13 / 18	Repair object ownership/lifecycle; do not reuse a freed handle.
TLS_FAILURE / X509_FAILURE	16 / 17	Report failed network/PKI validation without leaking secrets.

Table 6.1 Selected public status mapping

6.3 Ownership and secure clearing

The _native.handles.NativeHandle abstraction owns QSCABI-created handles and calls the matching native destructor in close(). The context-manager protocol uses __enter__/__exit__ for deterministic closure; a destructor is a fallback and must not be used as the only release plan. The issuer object must outlive a native certificate or CRL builder that borrows it. Pointer-valued properties are for internal wrapper calls, not a promise of independently transferable ownership.

qsc.memory.clear and other buffer helpers operate on caller-provided writable bytes. Secret-bearing copies should be explicitly cleared where feasible, and no native structure should be accessed after close. Passing memoryview over temporary buffers requires keeping the underlying owner alive for the native call. Opaque stream contexts are single-thread owned, even though the loader itself uses a lock.

7: Version, Metadata and ABI Qualification

7.1 Metadata API

qsc.version exposes major/minor/patch, ABI and QSC version strings, build platform/compiler and enabled feature descriptions. qsc.abi010 adds the ABI 0.10 compatibility predicate, uint64 capability mask, native backend and QSC build profile. A returned CPU feature string or capability flag is diagnostic metadata; it neither proves hardware device presence nor cryptographic certification.

7.2 Strict compatibility preflight

```
from qsc import abi010

info = abi010.require_full_abi()  # checks 0.10 compatibility and all 329 exports
print(info.abi_major, info.abi_minor, info.qsc_profile)
print(hex(info.capabilities))
```

Older libraries may resolve older APIs while missing ABI 0.10 additions. Strict qualification rejects missing exports. For production packaging, verify both symbol presence and native behavior across the required operating system, compiler and CPU target. A preflight failure is not repaired by changing a Python function pointer to an unrelated native export.

8: Encoding and Secure Memory

8.1 Hex, Base64 and PEM

qsc.encoding implements encoded/decoded size calculation and hex/Base64/PEM convenience/try variants, using caller buffers where requested. Strings are not interchangeable with bytes: decode text using the specified encoding and reject malformed input. A PEM parser or certificate decoder is not equivalent to X.509 trust verification. Always distinguish capacity from exact written size.

8.2 Wiping and comparisons

qsc.memory.clear operates on mutable buffers and `are_equal` wraps native byte equality. The caller retains ownership of plaintext/key arrays and should erase private keys, KEM secrets and intermediate KDF material as soon as possible. A constant-time comparator cannot protect length-dependent application branching; reject incompatible lengths first. Use MACs or signatures, not simple digests, to authenticate messages.

9: Hash, XOF and Streaming Digests

9.1 One-shot hashes

qsc.hash supports one-shot SHA2-256/384/512 and SHA3-128/256/512 families with fixed-output size helpers. The return of a digest computation is a public fingerprint of

the supplied message and is not an authenticator. Native failure status and output capacity must be checked before consuming the bytes.

```
from qsc import hash
message = b"QSC wrapper example"
result = hash.sha2_256(message)
assert len(result) == hash.sha2_256_digest_size()
```

9.2 Extendable output

qsc.xof provides SHAKE128, SHAKE256 and SHAKE512 convenience and try calls. Output length is chosen by the caller and must be bounded according to the application protocol; use cSHAKE customization explicitly when domain separation is required. A SHAKE digest is not automatically a keyed MAC.

10: MAC and Key Derivation

10.1 Authentication primitives

qsc.mac exposes HMAC-SHA2, KMAC, KMAC-XOF, QMAC, Poly1305 and PBMAC1 functions and size queries. Use the correct key length and output size for the selected native construction. Poly1305 keys must not be reused as generic long-term MAC keys without a protocol-defined one-time key schedule. KMAC customization strings form part of the authentication input; mismatched customization fails verification.

10.2 HKDF and PBMAC1

qsc.kdf exposes HKDF-SHA2 derivation and PBMAC1 helpers. Salt, input key material, customization/info, output byte length and password iteration count are separate security parameters. Password material and native keys must be handled as sensitive; a high-entropy shared secret is not the same as an arbitrary low-entropy password. Application deployment policy sets and validates work factors.

11: Symmetric Encryption and Authenticated Decryption

11.1 Cipher families

qsc.symmetric wraps AES block and CTR modes, AES-GCM, ChaCha20, ChaCha20-Poly1305, CSX and RCS according to native QSCABI availability. Size helpers return keys,

tags and nonce requirements. Raw CTR/ChaCha are unauthenticated stream transforms; their ciphertext needs a separately authenticated protocol. Authenticated modes must verify tags before accepting plaintext.

11.2 Nonce, capacity and overlap

Do not reuse an AEAD key/nonce pair. The QSCABI documented decrypt paths validate the destination capacity and refuse to release authenticated plaintext on tag failure. Treat `try_*` status and the actual byte count as part of the integrity decision, not as optional diagnostics. Do not overlap input and output buffers unless the exact native function explicitly documents support.

12: Entropy and Random Services

12.1 Providers and availability

qsc.entropy supplies ACP/CSP entropy and RDP/CJP provider functions with available/size helpers. Availability and platform source are native facts and must be checked at runtime. The package does not expose an arbitrary QSC SECRAND state lifecycle. An unavailable source must not be replaced by predictable bytes or a generic noncryptographic Python random.Random instance.

12.2 Cryptographic policy

Use QSC cryptographic providers for key generation, and do not confuse successful allocation with successful entropy generation. System error handling should remain fail-closed for production keys, handshakes and signature randomness. The native CPU feature banner reports detected instructions and an eligible candidate package profile only; it cannot inspect the SIMD instruction path compiled into an already loaded QSCABI DLL.

13: Post-Quantum KEMs and Signatures

13.1 Native-supported KEM and signature families

qsc.pqc exposes ML-KEM, HQC, Classic McEliece and signature wrappers such as ML-DSA, Falcon and SLH-DSA, subject to the native ABI and QSC profile. Sizing functions and keypair/encapsulation dataclasses avoid hard-coded lengths. Decapsulation

requires separate identity authentication and transcript binding. An implicit-rejection KEM may return secret material even when malformed ciphertext was supplied.

13.2 Falcon project policy

The tests/manifest.py FALCON_POLICY states that Falcon wrapper functions remain available for ABI completeness, while operational examples and sign/verify tests are commented out until the project adopts a FIPS-certified Falcon profile. This is a project policy statement, not an independent external certification claim. The API appendix preserves the callable wrapper declarations without promoting them as operationally qualified.

14: Classical Public-Key Operations

14.1 Key agreement and signatures

qsc.asymmetric supplies ECDH/EDDH key agreements and ECDSA/EdDSA signatures with length queries and seeded or random key generation where the native interface supports them. Validate public-key encoding, key length, signature format and detached/attached message policy at every application boundary. A successful agreement produces an unauthenticated shared secret unless the encompassing protocol authenticates the peer.

14.2 Secret ownership

The wrapper returns keypairs in typed Python objects, and some secret fields are held as mutable bytearray. Erase private keys and shared secrets when no longer needed, including temporary copies. Python runtime garbage collection and local-variable scope do not guarantee erasure. Verify exact signature and signed-message output length and reject a failed verification result.

15: Explicit Algorithm Selection

15.1 Runtime selectors

qsc.explicit_kem, explicit_curve, explicit_signature and explicit_slhdsa expose optional ABI 0.10 algorithm selectors, exact native size-query records and selected operations. The exported selector enums live in qsc._native.types and are re-exported from the

package. Runtime availability is checked using native symbol existence and must not be guessed from an enum member name.

15.2 SLH-DSA context and prehash

The SLH-DSA explicit wrapper documents the six SHAKE variants and both pure/prehash modes. Its context is limited to 255 bytes; message, context, prehash choice and signature dimensions must agree during verification. Randomized signing obtains randomness from the configured native source. `SlhDsaKeyPair.private_key` is mutable and should be cleared after use.

16: Native Streaming State and Lifecycle

16.1 State model

Public class	Normal sequence	Critical condition
Digest	with Digest(alg) as d: update*; finalize once	No updates or second finalization after final result.
Xof	with Xof(alg) as x: absorb*; finalize; squeeze*	Absorb only before finalize; repeat squeeze while open.
Mac	with Mac(alg,key) as m: update*; finalize	Secret key retained by native context until close.
Ctr	with Ctr(key,nonce,...) as c: transform*	Counter reuse forbidden; transform provides no authentication.

Table 16.1 QSCABI streaming wrappers

qsc.stream classes inherit private _NativeState, which owns a native Handle and records closed/finalized status. Every state is single-thread owned. Reusing a closed handle, double-finalizing, or squeezing before XOF finalization must be rejected by Python. Context managers call close on all paths including exceptions; do not rely on garbage collection for native destructor invocation.

16.2 Lifetime examples

```
from qsc.stream import Digest
from qsc._native.types import DigestAlgorithm

with Digest(DigestAlgorithm.SHA256) as digest:
    digest.update(b"chunk A")
    digest.update(b"chunk B")
    output = digest.finalize()
# Native digest state is closed here.
```

This example illustrates object lifecycle from the public source. Production code should test the actual DigestAlgorithm member available in the uploaded enums and handle native errors. A streaming digest does not authenticate messages.

17: X.509 Certificates, Chains and Trust Stores

17.1 Objects and conversion

qsc.x509 provides X509Certificate, X509Chain and X509TrustStore objects with `load_memory`, PEM/DER loaders, subject/issuer information, DNS fields, chain access, trust-store updates and validation methods. Objects own native handles, and the underlying QSCABI certificate loader copies input-backed bytes when required for parser lifetime. Output strings and byte buffers are caller-owned Python results; raw Handle properties remain borrowed.

17.2 Trust is separate from parsing

A syntactically valid certificate is not trusted by default. Build a chain to an independently trusted anchor, enforce expected hostname, validity dates, certificate usage constraints and revocation policy. The native wrapper's status values distinguish X.509 failures and invalid input. Revocation UNKNOWN/error must never silently become a GOOD/NOT_REVOKED status. Release trust stores and chains with deterministic context managers.

18: CSR Issuance, DER Signing and CRLs

18.1 CSR and CRL builder objects

qsc.issuance supplies X509Csr, X509CrlBuilder, P256DerSigner and verify_crl_der. These are optional ABI 0.10 X.509 issuance functions for P-256 CSRs, TLS certificate issuance and issuer-bound CRLs. An issued certificate remains subject to CA authorization, subject validation, serial management, signing-key custody and relying-party trust checks.

18.2 Synchronous callback security

P256DerSigner adapts a caller-supplied Python Callable[[bytes], bytes] to the synchronous native X509SignCallback. It copies bounded TBS bytes, requires DER ECDSA-P256-SHA256 signature output, checks the signature capacity, catches callback

exceptions before returning into native code, and raises a captured error after the native call completes. Keep the callback and issuer certificate alive until native use is complete; never return a pointer to a transient Python object.

The source enforces a 1 MiB bound on signed DER output and requires a 64-byte P-256 X||Y public key during CSR initialization. This describes the uploaded wrapper's constraints, not a general X.509 standard limit. A signed request proves possession of the signing key but does not authorize a DNS name or issuance purpose.

19: TLS 1.3 Context, Connection and Server APIs

19.1 Public object graph

qsc.tls defines TlsOptions, TlsSessionTicketPolicy, TlsPeerInformation and owned TlsContext, TlsConnection, TlsListener and TlsServer handles. TLS properties include policies, groups, signature preferences, trust paths, identity files, ALPN/SNI choices, client authentication and session tickets. The native connection is created, configured and handshaken only through its explicit lifecycle methods; constructing a Python TlsConnection alone does not authenticate a peer.

```
TlsContext.create(...) -> configure trust/identity/policy
TlsConnection.create() -> connect_host(ctx,hostname,service)
 -> handshake/peer verification -> send/receive
 -> shutdown -> close

TlsContext + configured identity -> TlsListener/TlsServer
 -> bind/start -> accept/serve -> close
```

19.2 Socket ownership and I/O status

The Python surface uses native QSCABI TLS connection handles instead of exposing QSC socket structs. `send` returns an accepted byte count; `receive` obtains caller-requested bytes; framed send/receive provide higher-level message boundaries. Partial operations, timeouts, peer shutdown, cancel, failed authentication and resource ownership must be managed explicitly. ALPN is a negotiated value only after a completed handshake.

19.3 ABI 0.10 preference/identity additions

The wrapper includes `TlsContext` additions for cipher-suite/group/signature preferences, trusted CA/CRL files, server and client certificate/key files, SNI identities, ALPN and session tickets. Where QSCABI marks an older in-memory loader as unsupported, use the supported file-based loader and do not replace path-based credentials with unverified memory-address tricks. Filenames and access permissions remain a deployment responsibility.

20: Hardware Provider Bridge

20.1 Status reporting and integration boundary

qsc.hardware currently exposes `available()` and `enabled()` to report the presence of the native hardware symbol and compiled provider support. The Python package has low-level qsc._native.hardware_native bindings for callback/handle operations, but it does not expose a full safe Python hardware-key object API in qsc.hardware. Do not imply that Python applications can provision, sign or attest with a physical HSM solely by invoking the two convenience functions.

20.2 No physical device assertion

Compile-time hardware support is not evidence of a detected token, TPM, SE050, secure element or independently verified non-exportability. A production feature requiring hardware key custody must validate provider callback lifetimes, object access, signature formats, device disconnection, attestation trust and failure behavior on the target equipment.

21: Python Integration Examples

21.1 Opt-in console workflow

The examples project uses a canonical ExampleGroup manifest with 15 entries. It provides an interactive menu and explicit `-run <group>` / `-all` dispatch. The console does not generate long-term keys or run authenticated network operations automatically on import. Missing native dependencies are displayed as failures/skip information rather than misleading PASS results.

Group	Purpose
version	QSCABI and linked QSC version metadata

Group	Purpose
encoding	hex, Base64, and PEM encoding helpers
hash	SHA2 and SHA3 digest helpers
xof	SHAKE XOF output helpers
mac	HMAC, KMAC, QMAC, Poly1305, and PBMAC1 helpers
kdf	HKDF and PBMAC1 key-derivation helpers
entropy	ACP and CSP entropy-provider helpers
symmetric	AES, ChaCha, CSX, and RCS symmetric examples
pqc	ML-KEM, HQC, McEliece, ML-DSA, and SLH-DSA examples
asymmetric	ECDH, EDDH, ECDSA, and EdDSA examples
x509	X.509 certificate, chain, hostname, and trust-store examples
tls	TLS context, option, ticket-policy, and handle lifecycle examples
abi010	QSCABI 0.10 native build/profile and 329 exported functions
tls010	TLS 1.3 preference IDs, trusted file and identity configuration
issuance	P-256 CSR preparation, signing boundary and CRL builder

Table 21.1 Source example console groups

21.2 Recommended implementation patterns

- Run ABI compatibility and loader checks before constructing native crypto or TLS objects.
- Use package-level size queries rather than baked-in key and ciphertext lengths.
- Prefer context managers for native TLS, X.509, streaming, CSR and CRL objects.
- Use `try_*` variants only when the caller explicitly handles both status and output length.
- Keep private keys and shared secrets in mutable buffers where possible and erase them promptly.
- Treat examples as demonstrations, not as evidence of device, peer or compliance qualification.

22: Test Harness and Qualification Evidence

22.1 22 test sections and 148 declared cases

tests/run_tests.py executes 22 named unittest modules in a fixed direct-call order. Each test method is reported with PASS/FAIL/SKIP and failures display traceback diagnostics. The default Visual Studio launch runs the full suite; `-test`, `-run`, `-list`, `-interactive`, `--strict-abi` and `--no-pause` support CI and targeted development. A console key prompt is only offered at the end when interactive stdin is attached and pausing was not disabled.

Test module	Declared methods
test_abi010_metadata.py	9
test_abi010_tls.py	6
test_abi010_x509.py	11
test_abi10_explicit.py	14
test_abi10_hardware.py	5
test_abi10_stream.py	12
test_asymmetric.py	7
test_core.py	3
test_encoding.py	4
test_entropy.py	3
test_hash.py	4
test_kdf.py	4
test_mac.py	6
test_native_bindings.py	5
test_native_integration.py	9
test_pqc.py	7
test_status.py	2
test_suite_consistency.py	6
test_symmetric.py	11
test_tls.py	10
test_x509.py	8
test_xof.py	2

Table 22.1 Uploaded test inventory

22.2 Qualification interpretation

The suite explicitly distinguishes skipped native-dependent tests from a successful cryptographic qualification. Strict ABI mode rejects any native skips or other failures and preflights all 329 exports. `QualificationResult` also distinguishes expected failures and unexpected successes. A source method named test_* counts as a declared case, not an executed PASS. The previous project chat includes historical runs, but no logs from that runtime are included in this ZIP.

22.3 Native and test distribution limitations

This paper did not execute the bundled QSCABI.dll or any Python test or example against a native QSC library. The bundled DLL is Windows x64 and cannot establish Linux, macOS, ARM or other CPU-profile qualification. The source-only checks performed for this specification were Python AST parsing, exported name registry

comparison and document rendering; these are not runtime cryptographic or network tests.

23: Security, Compatibility and Release Requirements

23.1 Primary security review

- Native function signatures, structures and c_size_t widths must match the loaded binary exactly. An incorrect ctypes type can cause memory corruption outside Python exception handling.
- Buffer creation and output length checks must precede any conversion of native bytes to an application result. Do not use invalid or partial plaintext, keys, tags, or certificate records.
- TLS certificate verification and hostname checking are separate from TCP connection creation and from SNI selection.
- No KEM result authenticates a peer without additional verified identity and transcript binding.
- Nonce/counter uniqueness, MAC domain separation and authenticated decryption failures must be explicitly tested.
- Python garbage collection is not a secret-wiping or secure native object-lifetime guarantee; close native handles deterministically.
- Falcon implementation exposure is distinct from the project policy on operational examples and tests.
- Compile-time capability flags, CPU detection, a bundled DLL, and syntax tests do not constitute security certification.

23.2 Supported distribution and ABI check

The implementor must pin the exact QSCABI native build and its linked QSC dependency, archive the OS/architecture/ISA profile, and run strict ABI + functional qualification on each target platform. The delivered Python package requires a proper QRCS-CASL license notice, native dependency documentation, and an authenticated distribution mechanism. The uploaded source does not include a pip package manifest or a release signing report.

24: Complete Public Python API Reference

This section reproduces 462 named public Python module functions, class methods and properties from the 24 qsc/*.py API modules. The total includes 19 @property descriptors; it excludes internal underscore names and special constructors/dunder protocol methods, which are covered by the class lifecycle and annex material. Signatures are derived from Python AST, preserving source type annotations and defaults. Brief descriptions are the first source docstring paragraph. Actual native behavior and permission to use an algorithm depend on its linked QSCABI implementation.

24.1 qsc.abi010 (7 Python entries)

Implementation source: qsc/abi010.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

abi_is_compatible

abi_is_compatible(major: int=0, minimum_minor: int=10) -> bool

Description: Check native ABI major/minor compatibility without accepting negative IDs.

capability_flags

capability_flags() -> int

Description: Return the complete unsigned 64-bit compile-time native capability mask.

build_backend

build_backend() -> str

Description: Return the native library's build-backend identifier string.

qsc_build_profile

qsc_build_profile() -> str

Description: Return the actual QSC build profile, not CPU feature detection.

build_info

build_info() -> NativeBuildInfo

Description: Read all available current native build metadata without mutating state.

missing_exports

missing_exports() -> tuple[str, ...]

Description: List every symbol absent from the currently loaded native library.

Notes: This loads the DLL and inspects symbol presence only. It cannot establish correct native implementation, signature validity or qualified hardware.

require_full_abi

require_full_abi() -> NativeBuildInfo

Description: Require 0.10 ABI compatibility and every one of 329 public exports.

Notes: Raise on incompatible version or any missing symbol; no SKIP is reported as a successful full-ABI qualification.

24.2 qsc.asymmetric (54 Python entries)

Implementation source: qsc/asymmetric.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

ecdh_public_key_size

ecdh_public_key_size() -> int

Description: Return the ECDH public-key size.

ecdh_private_key_size

ecdh_private_key_size() -> int

Description: Return the ECDH private-key size.

ecdh_shared_secret_size

ecdh_shared_secret_size() -> int

Description: Return the ECDH shared-secret size.

ecdh_seed_size

ecdh_seed_size() -> int

Description: Return the ECDH deterministic seed size.

ecdh_generate_keypair

ecdh_generate_keypair() -> AsymmetricKeyPair

Description: Generate an ECDH key pair.

try_ecdh_generate_keypair

try_ecdh_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an ECDH key pair into caller-owned buffers.

ecdh_generate_seeded_keypair

ecdh_generate_seeded_keypair(seed: bytes | bytearray | memoryview) -> AsymmetricKeyPair

Description: Generate a deterministic ECDH key pair from a seed.

try_ecdh_generate_seeded_keypair

try_ecdh_generate_seeded_keypair(seed: bytes | bytearray | memoryview, public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate a deterministic ECDH key pair into caller-owned buffers.

ecdh_public_from_private

ecdh_public_from_private(private_key: bytes | bytearray | memoryview) -> bytes

Description: Derive an ECDH public key from a private key.

try_ecdh_public_from_private

try_ecdh_public_from_private(private_key: bytes | bytearray | memoryview, public_key: bytearray | memoryview) -> QscStatus

Description: Derive an ECDH public key into caller-owned storage.

ecdh_key_exchange

ecdh_key_exchange(private_key: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Compute an ECDH shared secret.

try_ecdh_key_exchange

try_ecdh_key_exchange(private_key: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Compute an ECDH shared secret into caller-owned storage.

eddh_public_key_size

eddh_public_key_size() -> int

Description: Return the EDDH public-key size.

eddh_private_key_size

eddh_private_key_size() -> int

Description: Return the EDDH private-key size.

eddh_shared_secret_size

eddh_shared_secret_size() -> int

Description: Return the EDDH shared-secret size.

eddh_seed_size

eddh_seed_size() -> int

Description: Return the EDDH deterministic seed size.

eddh_generate_keypair

eddh_generate_keypair() -> AsymmetricKeyPair

Description: Generate an EDDH key pair.

try_eddh_generate_keypair

try_eddh_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an EDDH key pair into caller-owned buffers.

eddh_generate_seeded_keypair

eddh_generate_seeded_keypair(seed: bytes | bytearray | memoryview) -> AsymmetricKeyPair

Description: Generate a deterministic EDDH key pair from a seed.

try_eddh_generate_seeded_keypair

try_eddh_generate_seeded_keypair(seed: bytes | bytearray | memoryview, public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate a deterministic EDDH key pair into caller-owned buffers.

eddh_public_from_private

eddh_public_from_private(private_key: bytes | bytearray | memoryview) -> bytes

Description: Derive an EDDH public key from a private key.

try_eddh_public_from_private

try_eddh_public_from_private(private_key: bytes | bytearray | memoryview, public_key: bytearray | memoryview) -> QscStatus

Description: Derive an EDDH public key into caller-owned storage.

eddh_key_exchange

eddh_key_exchange(private_key: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Compute an EDDH shared secret.

try_eddh_key_exchange

try_eddh_key_exchange(private_key: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Compute an EDDH shared secret into caller-owned storage.

ecdsa_public_key_size

ecdsa_public_key_size() -> int

Description: Return the ECDSA public-key size.

ecdsa_private_key_size

ecdsa_private_key_size() -> int

Description: Return the ECDSA private-key size.

ecdsa_signature_size

ecdsa_signature_size() -> int

Description: Return the ECDSA detached signature size.

ecdsa_seed_size

ecdsa_seed_size() -> int

Description: Return the ECDSA deterministic seed size.

ecdsa_signed_message_size

ecdsa_signed_message_size(message_length: int) -> int

Description: Return the ECDSA signed-message size for a message length.

ecdsa_generate_keypair

ecdsa_generate_keypair() -> AsymmetricKeyPair

Description: Generate an ECDSA key pair.

try_ecdsa_generate_keypair

try_ecdsa_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an ECDSA key pair into caller-owned buffers.

ecdsa_generate_seeded_keypair

ecdsa_generate_seeded_keypair(seed: bytes | bytearray | memoryview) -> AsymmetricKeyPair

Description: Generate a deterministic ECDSA key pair from a seed.

try_ecdsa_generate_seeded_keypair

try_ecdsa_generate_seeded_keypair(seed: bytes | bytearray | memoryview, public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate a deterministic ECDSA key pair into caller-owned buffers.

ecdsa_public_from_private

ecdsa_public_from_private(private_key: bytes | bytearray | memoryview) -> bytes

Description: Derive an ECDSA public key from a private key.

try_ecdsa_public_from_private

try_ecdsa_public_from_private(private_key: bytes | bytearray | memoryview, public_key: bytearray | memoryview) -> QscStatus

Description: Derive an ECDSA public key into caller-owned storage.

ecdsa_sign

ecdsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Sign a message with ECDSA and return QSCABI signed-message bytes.

try_ecdsa_sign

try_ecdsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, signed_message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Sign an ECDSA message into caller-owned storage.

ecdsa_verify

ecdsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Verify an ECDSA signed message and recover the original message.

try_ecdsa_verify

try_ecdsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Verify an ECDSA signed message into caller-owned storage.

eddsa_public_key_size

eddsa_public_key_size() -> int

Description: Return the EdDSA public-key size.

eddsa_private_key_size

eddsa_private_key_size() -> int

Description: Return the EdDSA private-key size.

eddsa_signature_size

eddsa_signature_size() -> int

Description: Return the EdDSA detached signature size.

eddsa_seed_size

eddsa_seed_size() -> int

Description: Return the EdDSA deterministic seed size.

eddsa_signed_message_size

eddsa_signed_message_size(message_length: int) -> int

Description: Return the EdDSA signed-message size for a message length.

eddsa_generate_keypair

eddsa_generate_keypair() -> AsymmetricKeyPair

Description: Generate an EdDSA key pair.

try_eddsa_generate_keypair

try_eddsa_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an EdDSA key pair into caller-owned buffers.

eddsa_generate_seeded_keypair

eddsa_generate_seeded_keypair(seed: bytes | bytearray | memoryview) -> AsymmetricKeyPair

Description: Generate a deterministic EdDSA key pair from a seed.

try_eddsa_generate_seeded_keypair

try_eddsa_generate_seeded_keypair(seed: bytes | bytearray | memoryview, public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate a deterministic EdDSA key pair into caller-owned buffers.

eddsa_public_from_private

eddsa_public_from_private(private_key: bytes | bytearray | memoryview) -> bytes

Description: Derive an EdDSA public key from a private key.

try_eddsa_public_from_private

try_eddsa_public_from_private(private_key: bytes | bytearray | memoryview, public_key: bytearray | memoryview) -> QscStatus

Description: Derive an EdDSA public key into caller-owned storage.

eddsa_sign

eddsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Sign a message with EdDSA and return QSCABI signed-message bytes.

try_eddsa_sign

try_eddsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, signed_message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Sign an EdDSA message into caller-owned storage.

eddsa_verify

eddsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Verify an EdDSA signed message and recover the original message.

try_eddsa_verify

try_eddsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Verify an EdDSA signed message into caller-owned storage.

24.3 qsc.buffers (2 Python entries)

Implementation source: qsc/buffers.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

require_bytes

require_bytes(data: bytes | bytearray | memoryview, name: str='data') -> bytes

Description: Return a bytes copy from a bytes-like object.

require_writable_byte_buffer

require_writable_byte_buffer(buffer: bytearray | memoryview, name: str='buffer') -> memoryview

Description: Return a writable one-dimensional byte memoryview.

24.4 qsc.encoding (16 Python entries)

Implementation source: qsc/encoding.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

hex_encoded_size

hex_encoded_size(input_length: int) -> int

Description: Return the number of hexadecimal characters needed for input_length bytes.

hex_decoded_size

hex_decoded_size(text: str | bytes | bytearray | memoryview) -> int

Description: Return the binary byte count represented by a hexadecimal string.

hex_encode

hex_encode(data: bytes | bytearray | memoryview) -> str

Description: Encode bytes as lower-case hexadecimal text.

try_hex_encode

try_hex_encode(output: bytearray | memoryview, data: bytes | bytearray | memoryview) -> tuple[QscStatus, int]

Description: Encode hexadecimal text into caller-owned storage and return status/written.

hex_decode

hex_decode(text: str | bytes | bytearray | memoryview) -> bytes

Description: Decode hexadecimal text into bytes.

try_hex_decode

try_hex_decode(output: bytearray | memoryview, text: str | bytes | bytearray | memoryview) -> tuple[QscStatus, int]

Description: Decode hexadecimal text into caller-owned storage and return status/written.

base64_encoded_size

base64_encoded_size(input_length: int) -> int

Description: Return the number of Base64 characters needed for input_length bytes.

base64_decoded_size

base64_decoded_size(text: str | bytes | bytearray | memoryview) -> int

Description: Return the binary byte count represented by a Base64 string.

base64_encode

base64_encode(data: bytes | bytearray | memoryview) -> str

Description: Encode bytes as RFC 4648 Base64 text.

try_base64_encode

try_base64_encode(output: bytearray | memoryview, data: bytes | bytearray | memoryview) -> tuple[QscStatus, int]

Description: Encode Base64 text into caller-owned storage and return status/written.

base64_decode

base64_decode(text: str | bytes | bytearray | memoryview) -> bytes

Description: Decode RFC 4648 Base64 text into bytes.

try_base64_decode

try_base64_decode(output: bytearray | memoryview, text: str | bytes | bytearray | memoryview) -> tuple[QscStatus, int]

Description: Decode Base64 text into caller-owned storage and return status/written.

pem_encoded_size

pem_encoded_size(label: str | bytes, data_length: int) -> int

Description: Return the number of PEM characters required for a label and DER length.

pem_decoded_size

pem_decoded_size(text: str | bytes | bytearray | memoryview) -> int

Description: Return the decoded DER byte count represented by a PEM document.

pem_encode

pem_encode(label: str | bytes, data: bytes | bytearray | memoryview) -> str

Description: Encode DER bytes as a PEM document.

pem_decode

pem_decode(text: str | bytes | bytearray | memoryview) -> bytes

Description: Decode a PEM document into DER bytes.

24.5 qsc.entropy (14 Python entries)

Implementation source: qsc/entropy.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

acp_seed_max

acp_seed_max() -> int

Description: Return the maximum ACP output length advertised by the ABI.

csp_seed_max

csp_seed_max() -> int

Description: Return the maximum CSP output length advertised by the ABI.

rdp_seed_max

rdp_seed_max() -> int

Description: Return the maximum RDP output length advertised by the ABI.

cjp_seed_max

cjp_seed_max() -> int

Description: Return the maximum CJP output length advertised by the ABI.

rdp_available

rdp_available() -> bool

Description: Return True when the RDP provider is available on the current platform.

cjp_available

cjp_available() -> bool

Description: Return True when the CJP provider is available on the current platform.

try_acp_generate

try_acp_generate(output: bytearray | memoryview) -> QscStatus

Description: Fill caller-owned storage from the ACP provider and return status.

try_csp_generate

try_csp_generate(output: bytearray | memoryview) -> QscStatus

Description: Fill caller-owned storage from the CSP provider and return status.

try_rdp_generate

try_rdp_generate(output: bytearray | memoryview) -> QscStatus

Description: Fill caller-owned storage from the RDP provider and return status.

try_cjp_generate

try_cjp_generate(output: bytearray | memoryview) -> QscStatus

Description: Fill caller-owned storage from the CJP provider and return status.

acp_generate

acp_generate(length: int) -> bytes

Description: Generate bytes using the platform ACP provider.

csp_generate

csp_generate(length: int) -> bytes

Description: Generate bytes using the operating-system cryptographic provider.

rdp_generate

rdp_generate(length: int) -> bytes

Description: Generate bytes using the hardware RDP provider when available.

cjp_generate

cjp_generate(length: int) -> bytes

Description: Generate bytes using the CPU-jitter provider when available.

24.6 qsc.errors (1 Python entries)

Implementation source: qsc/errors.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

ensure_success

ensure_success(status: int | QscStatus, operation: str | None=None) -> None

Description: Raise QscError when the supplied status is not QSCABI success.

24.7 qsc.explicit_curve (5 Python entries)

Implementation source: qsc/explicit_curve.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

available

available() -> bool

Description: Report whether this DLL exposes the newer explicit curve-size function.

sizes

sizes(algorithm: CurveAlgorithm) -> CurveSizes

Description: Query native public/private/secret/seed bytes; zero seed is allowed.

generate

generate(algorithm: CurveAlgorithm, seed: bytes | bytearray | memoryview | None=None) -> CurveKeyPair

Description: Generate a fresh keypair or derive one from an explicitly supplied seed.

Notes: A seeded pair is deterministic and therefore unsafe with predictable seeds. Never reuse illustrative seeds for real private keys.

public_from_private

public_from_private(algorithm: CurveAlgorithm, private_key: bytes | bytearray | memoryview) -> bytes

Description: Compute a public point from an exact-sized, caller-owned private key.

agree

agree(algorithm: CurveAlgorithm, peer_public_key: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytearray

Description: Return a secret only after the native key agreement succeeds.

Notes: Does not provide authentication, traffic-key derivation or replay defense.

24.8 qsc.explicit_kem (5 Python entries)

Implementation source: qsc/explicit_kem.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

available

available() -> bool

Description: Check for the named KEM sizes export in the loaded DLL, if any.

sizes

sizes(algorithm: KemAlgorithm) -> KemSizes

Description: Query public/private/ciphertext/secret byte sizes for the selected KEM.

Notes: Raises QscError for unsupported algorithms or malformed native sizes.

generate

generate(algorithm: KemAlgorithm) -> KemKeyPair

Description: Generate a keypair; private key is mutable and remains caller-owned.

encapsulate

encapsulate(algorithm: KemAlgorithm, public_key: bytes | bytearray | memoryview) -> KemEncapsulation

Description: Produce ciphertext and a secret for a caller-authenticated public key.

decapsulate

decapsulate(algorithm: KemAlgorithm, ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytearray

Description: Decapsulate only after exact input-length validation and native success.

24.9 qsc.explicit_signature (5 Python entries)

Implementation source: qsc/explicit_signature.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

available

available() -> bool

Description: Check for explicit attached-signature size metadata in the DLL.

sizes

sizes(algorithm: SignatureAlgorithm) -> SignatureSizes

Description: Read native public/private, maximum prefix, and seed capacities.

generate

generate(algorithm: SignatureAlgorithm) -> SignatureKeyPair

Description: Generate a native signing keypair; protect and clear the private output.

sign

sign(algorithm: SignatureAlgorithm, message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Return signed message, bounded by max signature prefix plus message.

Notes: `message` and `private_key` are borrowed, and signed bytes remain unauthenticated until a matching public-key verification succeeds.

verify

verify(algorithm: SignatureAlgorithm, signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Verify and recover the original message; raise if native verification fails.

Notes: Never trust recovered bytes after a non-success native status.

24.10 qsc.explicit_slhdsa (5 Python entries)

Implementation source: qsc/explicit_slhdsa.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

available

available() -> bool

Description: Check for the explicit SLH-DSA size symbol in this native DLL.

sizes

sizes(algorithm: SlhDsaAlgorithm) -> SlhDsaSizes

Description: Return exact public/private/signature byte counts plus optional seed.

generate

generate(algorithm: SlhDsaAlgorithm) -> SlhDsaKeyPair

Description: Generate a native SLH-DSA keypair with the selected parameter set.

sign

sign(algorithm: SlhDsaAlgorithm, prehash: SlhDsaPrehash, message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, *, context: bytes | bytearray | memoryview=b'', randomized: bool=False) -> bytes

Description: Return a detached signature; native randomized signing uses native entropy.

verify

verify(algorithm: SlhDsaAlgorithm, prehash: SlhDsaPrehash, signature: bytes | bytearray | memoryview, message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, *, context: bytes | bytearray | memoryview=b'') -> bool

Description: Verify a detached signature; raise QscError on verification failure.

Notes: Returning True means valid under supplied public key, NOT trusted identity.

24.11 qsc.hardware (2 Python entries)

Implementation source: qsc/hardware.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

available

available() -> bool

Description: Return True when the optional hardware-provider export is discoverable.

Notes: Symbol presence alone does not establish any connected hardware or policy.

enabled

enabled() -> bool

Description: Ask native QSCABI if hardware API support was enabled in this build.

Notes: This is not a device health check, key-custody guarantee or FIPS claim. A DLL lacking this newer symbol reports False without failing imports.

24.12 qsc.hash (18 Python entries)

Implementation source: qsc/hash.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

sha2_256_digest_size

sha2_256_digest_size() -> int

Description: Return the SHA2-256 digest size in bytes.

sha2_384_digest_size

sha2_384_digest_size() -> int

Description: Return the SHA2-384 digest size in bytes.

sha2_512_digest_size

sha2_512_digest_size() -> int

Description: Return the SHA2-512 digest size in bytes.

sha3_128_digest_size

sha3_128_digest_size() -> int

Description: Return the QSC SHA3-128 digest size in bytes.

sha3_256_digest_size

sha3_256_digest_size() -> int

Description: Return the SHA3-256 digest size in bytes.

sha3_512_digest_size

sha3_512_digest_size() -> int

Description: Return the SHA3-512 digest size in bytes.

sha2_256

sha2_256(message: bytes | bytearray | memoryview) -> bytes

Description: Compute a SHA2-256 digest.

sha2_384

sha2_384(message: bytes | bytearray | memoryview) -> bytes

Description: Compute a SHA2-384 digest.

sha2_512

sha2_512(message: bytes | bytearray | memoryview) -> bytes

Description: Compute a SHA2-512 digest.

sha3_128

sha3_128(message: bytes | bytearray | memoryview) -> bytes

Description: Compute a QSC SHA3-128 digest.

sha3_256

sha3_256(message: bytes | bytearray | memoryview) -> bytes

Description: Compute a SHA3-256 digest.

sha3_512

sha3_512(message: bytes | bytearray | memoryview) -> bytes

Description: Compute a SHA3-512 digest.

try_sha2_256

try_sha2_256(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHA2-256 into caller-owned storage.

try_sha2_384

try_sha2_384(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHA2-384 into caller-owned storage.

try_sha2_512

try_sha2_512(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHA2-512 into caller-owned storage.

try_sha3_128

try_sha3_128(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHA3-128 into caller-owned storage.

try_sha3_256

try_sha3_256(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHA3-256 into caller-owned storage.

try_sha3_512

try_sha3_512(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHA3-512 into caller-owned storage.

24.13 qsc.issuance (15 Python entries)

Implementation source: qsc/issuance.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

P256DerSigner.callback [property]

callback(self) -> X509SignCallback

Description: Return the C function pointer while this signer object is alive.

X509Csr.handle [property]

handle(self) -> Handle

Description: Borrow the live opaque native handle for another synchronous call.

X509Csr.init_p256

init_p256(self, common_name: str, public_key: bytes | bytearray | memoryview) -> None

Description: Initialize 64-byte P-256 X||Y public key and a validated common name.

X509Csr.add_dns

add_dns(self, dns: str) -> None

Description: Add a DNS Subject Alternative Name; native X.509 validates format.

X509Csr.load_der

load_der(self, der: bytes | bytearray | memoryview) -> None

Description: Decode a signed DER CSR; caller retains the original input bytes.

X509Csr.verify

verify(self) -> None

Description: Verify CSR proof-of-possession signature, *not* domain authorization.

X509Csr.sign_der

sign_der(self, signer: P256DerSigner, capacity: int=16384) -> bytes

Description: Sign a prepared CSR using a caller-managed synchronous DER signer.

X509Csr.issue_tls_server_p256

issue_tls_server_p256(self, issuer, serial, not_before, not_after, signer: P256DerSigner, capacity: int=16384) -> bytes

Description: Issue a P-256 TLS server certificate from signed CSR and trusted CA.

Notes: The caller must authorize the requested identities and verify issuer possession, CA authority, serial uniqueness, and certificate validity. A generated certificate is not automatically a trusted certificate.

X509Csr.close

close(self) -> None

Description: Destroy native CSR state once; using the closed object raises.

X509CrlBuilder.handle [property]

handle(self) -> Handle

Description: Borrow the live CRL builder handle; never free it with Python allocators.

X509CrlBuilder.set_number

set_number(self, value: bytes | bytearray | memoryview) -> None

Description: Set a positive, unique big-endian CRLNumber extension value.

X509CrlBuilder.add_revoked

add_revoked(self, serial: bytes | bytearray | memoryview, when) -> None

Description: Add a revoked serial with a validated UTC revocation time.

X509CrlBuilder.sign_der

sign_der(self, signer: P256DerSigner, capacity: int=16384) -> bytes

Description: DER-sign CRL with the issuer's external private-key callback.

Notes: The issuer certificate must remain live, and the caller must verify issuer trust, signature, and revocation semantics before publishing.

X509CrlBuilder.close

close(self) -> None

Description: Destroy the native CRL before releasing reference to its issuer.

verify_crl_der

verify_crl_der(der, issuer, at) -> None

Description: Verify signed CRL DER against an authorized issuer and UTC time.

Notes: Caller remains responsible for the issuer chain, expected EKU/role, certificate trust and revocation policy. No native handles are retained.

24.14 qsc.kdf (19 Python entries)

Implementation source: qsc/kdf.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

hkdf_sha2_256_prk_size

hkdf_sha2_256_prk_size() -> int

Description: Return the HKDF-SHA2-256 PRK size.

hkdf_sha2_384_prk_size

hkdf_sha2_384_prk_size() -> int

Description: Return the HKDF-SHA2-384 PRK size.

hkdf_sha2_512_prk_size

hkdf_sha2_512_prk_size() -> int

Description: Return the HKDF-SHA2-512 PRK size.

pbmac1_max_key_size

pbmac1_max_key_size() -> int

Description: Return the maximum PBMAC1 derived-key size.

pbmac1_max_iterations

pbmac1_max_iterations() -> int

Description: Return the maximum accepted PBMAC1 iteration count.

hkdf_sha2_256_extract

hkdf_sha2_256_extract(key: bytes | bytearray | memoryview, salt: bytes | bytearray | memoryview=b'') -> bytes

Description: Run HKDF-SHA2-256 extract and return the PRK.

hkdf_sha2_384_extract

hkdf_sha2_384_extract(key: bytes | bytearray | memoryview, salt: bytes | bytearray | memoryview=b'') -> bytes

Description: Run HKDF-SHA2-384 extract and return the PRK.

hkdf_sha2_512_extract

hkdf_sha2_512_extract(key: bytes | bytearray | memoryview, salt: bytes | bytearray | memoryview=b'') -> bytes

Description: Run HKDF-SHA2-512 extract and return the PRK.

try_hkdf_sha2_256_extract

try_hkdf_sha2_256_extract(output, key, salt=b'') -> QscStatus

Description: Run HKDF-SHA2-256 extract into caller-owned storage.

try_hkdf_sha2_384_extract

try_hkdf_sha2_384_extract(output, key, salt=b'') -> QscStatus

Description: Run HKDF-SHA2-384 extract into caller-owned storage.

try_hkdf_sha2_512_extract

try_hkdf_sha2_512_extract(output, key, salt=b'') -> QscStatus

Description: Run HKDF-SHA2-512 extract into caller-owned storage.

hkdf_sha2_256_expand

hkdf_sha2_256_expand(prk, info=b'', output_length: int=32) -> bytes

Description: Run HKDF-SHA2-256 expand.

hkdf_sha2_384_expand

hkdf_sha2_384_expand(prk, info=b'', output_length: int=48) -> bytes

Description: Run HKDF-SHA2-384 expand.

hkdf_sha2_512_expand

hkdf_sha2_512_expand(prk, info=b'', output_length: int=64) -> bytes

Description: Run HKDF-SHA2-512 expand.

try_hkdf_sha2_256_expand

try_hkdf_sha2_256_expand(output, prk, info=b'') -> QscStatus

Description: Run HKDF-SHA2-256 expand into caller-owned storage.

try_hkdf_sha2_384_expand

try_hkdf_sha2_384_expand(output, prk, info=b'') -> QscStatus

Description: Run HKDF-SHA2-384 expand into caller-owned storage.

try_hkdf_sha2_512_expand

try_hkdf_sha2_512_expand(output, prk, info=b'') -> QscStatus

Description: Run HKDF-SHA2-512 expand into caller-owned storage.

pbmac1_derive_key

pbmac1_derive_key(hash_profile: QscPbmac1Hash | int, password, salt, iterations: int, key_length: int) -> bytes

Description: Derive a PBMAC1 key with the selected hash profile.

try_pbmac1_derive_key

try_pbmac1_derive_key(output, hash_profile: QscPbmac1Hash | int, password, salt, iterations: int, key_length: int | None=None) -> QscStatus

Description: Derive a PBMAC1 key into caller-owned storage.

24.15 qsc.mac (37 Python entries)

Implementation source: qsc/mac.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

hmac_sha2_256_mac_size

hmac_sha2_256_mac_size() -> int

Description: Return the HMAC-SHA2-256 output length.

hmac_sha2_384_mac_size

hmac_sha2_384_mac_size() -> int

Description: Return the HMAC-SHA2-384 output length.

hmac_sha2_512_mac_size

hmac_sha2_512_mac_size() -> int

Description: Return the HMAC-SHA2-512 output length.

poly1305_mac_size

poly1305_mac_size() -> int

Description: Return the Poly1305 output length.

poly1305_key_size

poly1305_key_size() -> int

Description: Return the Poly1305 key length.

qmac_mac_size

qmac_mac_size() -> int

Description: Return the QMAC output length.

qmac_key_size

qmac_key_size() -> int

Description: Return the QMAC key length.

qmac_nonce_size

qmac_nonce_size() -> int

Description: Return the QMAC nonce length.

pbmac1_mac_size

pbmac1_mac_size(hash_profile: QscPbmac1Hash | int) -> int

Description: Return the PBMAC1 MAC length for the selected hash profile.

hmac_sha2_256

hmac_sha2_256(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> bytes

Description: Compute an HMAC-SHA2-256 authentication code.

hmac_sha2_384

hmac_sha2_384(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> bytes

Description: Compute an HMAC-SHA2-384 authentication code.

hmac_sha2_512

hmac_sha2_512(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> bytes

Description: Compute an HMAC-SHA2-512 authentication code.

try_hmac_sha2_256

try_hmac_sha2_256(output: bytearray | memoryview, message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> QscStatus

Description: Compute HMAC-SHA2-256 into caller-owned storage.

try_hmac_sha2_384

try_hmac_sha2_384(output: bytearray | memoryview, message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> QscStatus

Description: Compute HMAC-SHA2-384 into caller-owned storage.

try_hmac_sha2_512

try_hmac_sha2_512(output: bytearray | memoryview, message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> QscStatus

Description: Compute HMAC-SHA2-512 into caller-owned storage.

kmac_128

kmac_128(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, custom: bytes | bytearray | memoryview=b'', output_length: int=32) -> bytes

Description: Compute a fixed-output KMAC-128 code.

kmac_256

kmac_256(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, custom: bytes | bytearray | memoryview=b'', output_length: int=64) -> bytes

Description: Compute a fixed-output KMAC-256 code.

kmac_512

kmac_512(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, custom: bytes | bytearray | memoryview=b'', output_length: int=64) -> bytes

Description: Compute a fixed-output KMAC-512 code.

kmac_xof_128

kmac_xof_128(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, custom: bytes | bytearray | memoryview, output_length: int) -> bytes

Description: Compute variable-length KMAC-XOF-128 output.

kmac_xof_256

kmac_xof_256(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, custom: bytes | bytearray | memoryview, output_length: int) -> bytes

Description: Compute variable-length KMAC-XOF-256 output.

kmac_xof_512

kmac_xof_512(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, custom: bytes | bytearray | memoryview, output_length: int) -> bytes

Description: Compute variable-length KMAC-XOF-512 output.

try_kmac_128

try_kmac_128(output, message, key, custom=b'') -> QscStatus

Description: Compute KMAC-128 into caller-owned storage.

try_kmac_256

try_kmac_256(output, message, key, custom=b'') -> QscStatus

Description: Compute KMAC-256 into caller-owned storage.

try_kmac_512

try_kmac_512(output, message, key, custom=b'') -> QscStatus

Description: Compute KMAC-512 into caller-owned storage.

try_kmac_xof_128

try_kmac_xof_128(output, message, key, custom=b'') -> QscStatus

Description: Compute KMAC-XOF-128 into caller-owned storage.

try_kmac_xof_256

try_kmac_xof_256(output, message, key, custom=b'') -> QscStatus

Description: Compute KMAC-XOF-256 into caller-owned storage.

try_kmac_xof_512

try_kmac_xof_512(output, message, key, custom=b'') -> QscStatus

Description: Compute KMAC-XOF-512 into caller-owned storage.

poly1305

poly1305(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> bytes

Description: Compute a Poly1305 authentication code.

try_poly1305

try_poly1305(output: bytearray | memoryview, message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> QscStatus

Description: Compute Poly1305 into caller-owned storage.

poly1305_verify

poly1305_verify(code: bytes | bytearray | memoryview, message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> bool

Description: Return True when the supplied Poly1305 code verifies.

try_poly1305_verify

try_poly1305_verify(code: bytes | bytearray | memoryview, message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview) -> QscStatus

Description: Verify a Poly1305 code and return the native status.

qmac

qmac(message: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, info: bytes | bytearray | memoryview=b'') -> bytes

Description: Compute a QMAC authentication code.

try_qmac

try_qmac(output, message, key, nonce, info=b'') -> QscStatus

Description: Compute QMAC into caller-owned storage.

pbmac1

pbmac1(hash_profile: QscPbmac1Hash | int, password, salt, iterations: int, key_length: int, message) -> bytes

Description: Compute a PBMAC1 authentication code.

try_pbmac1

try_pbmac1(output, hash_profile: QscPbmac1Hash | int, password, salt, iterations: int, key_length: int, message) -> QscStatus

Description: Compute PBMAC1 into caller-owned storage.

pbmac1_verify

pbmac1_verify(code, hash_profile: QscPbmac1Hash | int, password, salt, iterations: int, key_length: int, message) -> bool

Description: Return True when the supplied PBMAC1 code verifies.

try_pbmac1_verify

try_pbmac1_verify(code, hash_profile: QscPbmac1Hash | int, password, salt, iterations: int, key_length: int, message) -> QscStatus

Description: Verify a PBMAC1 code and return the native status.

24.16 qsc.memory (2 Python entries)

Implementation source: qsc/memory.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

clear

clear(buffer: bytearray | memoryview) -> None

Description: Clear a caller-owned mutable byte buffer in place.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

are_equal

are_equal(left: bytes | bytearray | memoryview, right: bytes | bytearray | memoryview) -> bool

Description: Compare two byte buffers using the QSCABI constant-time equality helper.

24.17 qsc.pqc (60 Python entries)

Implementation source: qsc/pqc.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

mlkem_public_key_size

mlkem_public_key_size() -> int

Description: Return the ML-KEM public-key size.

mlkem_private_key_size

mlkem_private_key_size() -> int

Description: Return the ML-KEM private-key size.

mlkem_ciphertext_size

mlkem_ciphertext_size() -> int

Description: Return the ML-KEM ciphertext size.

mlkem_shared_secret_size

mlkem_shared_secret_size() -> int

Description: Return the ML-KEM shared-secret size.

hqc_public_key_size

hqc_public_key_size() -> int

Description: Return the HQC public-key size.

hqc_private_key_size

hqc_private_key_size() -> int

Description: Return the HQC private-key size.

hqc_ciphertext_size

hqc_ciphertext_size() -> int

Description: Return the HQC ciphertext size.

hqc_shared_secret_size

hqc_shared_secret_size() -> int

Description: Return the HQC shared-secret size.

mceliece_public_key_size

mceliece_public_key_size() -> int

Description: Return the Classic McEliece public-key size.

mceliece_private_key_size

mceliece_private_key_size() -> int

Description: Return the Classic McEliece private-key size.

mceliece_ciphertext_size

mceliece_ciphertext_size() -> int

Description: Return the Classic McEliece ciphertext size.

mceliece_shared_secret_size

mceliece_shared_secret_size() -> int

Description: Return the Classic McEliece shared-secret size.

mlkem_generate_keypair

mlkem_generate_keypair() -> PqcKeyPair

Description: Generate an ML-KEM key pair.

try_mlkem_generate_keypair

try_mlkem_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an ML-KEM key pair into caller-owned buffers.

mlkem_encapsulate

mlkem_encapsulate(public_key: bytes | bytearray | memoryview) -> KemEncapsulation

Description: Encapsulate an ML-KEM shared secret to a public key.

try_mlkem_encapsulate

try_mlkem_encapsulate(public_key: bytes | bytearray | memoryview, ciphertext: bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Encapsulate an ML-KEM secret into caller-owned buffers.

mlkem_decapsulate

mlkem_decapsulate(ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Decapsulate an ML-KEM shared secret.

try_mlkem_decapsulate

try_mlkem_decapsulate(ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Decapsulate an ML-KEM secret into caller-owned storage.

hqc_generate_keypair

hqc_generate_keypair() -> PqcKeyPair

Description: Generate an HQC key pair.

try_hqc_generate_keypair

try_hqc_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an HQC key pair into caller-owned buffers.

hqc_encapsulate

hqc_encapsulate(public_key: bytes | bytearray | memoryview) -> KemEncapsulation

Description: Encapsulate an HQC shared secret to a public key.

try_hqc_encapsulate

try_hqc_encapsulate(public_key: bytes | bytearray | memoryview, ciphertext: bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Encapsulate an HQC secret into caller-owned buffers.

hqc_decapsulate

hqc_decapsulate(ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Decapsulate an HQC shared secret.

try_hqc_decapsulate

try_hqc_decapsulate(ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Decapsulate an HQC secret into caller-owned storage.

mceliece_generate_keypair

mceliece_generate_keypair() -> PqcKeyPair

Description: Generate a Classic McEliece key pair.

try_mceliece_generate_keypair

try_mceliece_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate a Classic McEliece key pair into caller-owned buffers.

mceliece_encapsulate

mceliece_encapsulate(public_key: bytes | bytearray | memoryview) -> KemEncapsulation

Description: Encapsulate a Classic McEliece shared secret to a public key.

try_mceliece_encapsulate

try_mceliece_encapsulate(public_key: bytes | bytearray | memoryview, ciphertext: bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Encapsulate a Classic McEliece secret into caller-owned buffers.

mceliece_decapsulate

mceliece_decapsulate(ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Decapsulate a Classic McEliece shared secret.

try_mceliece_decapsulate

try_mceliece_decapsulate(ciphertext: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, shared_secret: bytearray | memoryview) -> QscStatus

Description: Decapsulate a Classic McEliece secret into caller-owned storage.

mldsa_public_key_size

mldsa_public_key_size() -> int

Description: Return the ML-DSA public-key size.

mldsa_private_key_size

mldsa_private_key_size() -> int

Description: Return the ML-DSA private-key size.

mldsa_signature_size

mldsa_signature_size() -> int

Description: Return the ML-DSA detached signature size.

mldsa_signed_message_size

mldsa_signed_message_size(message_length: int) -> int

Description: Return the ML-DSA signed-message size for a message length.

mldsa_generate_keypair

mldsa_generate_keypair() -> PqcKeyPair

Description: Generate an ML-DSA key pair.

try_mldsa_generate_keypair

try_mldsa_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an ML-DSA key pair into caller-owned buffers.

mldsa_sign

mldsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Sign a message with ML-DSA and return QSCABI signed-message bytes.

try_mldsa_sign

try_mldsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, signed_message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Sign an ML-DSA message into caller-owned storage.

mldsa_verify

mldsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Verify an ML-DSA signed message and recover the original message.

try_mldsa_verify

try_mldsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Verify an ML-DSA signed message into caller-owned storage.

slhdsa_public_key_size

slhdsa_public_key_size() -> int

Description: Return the SLH-DSA public-key size.

slhdsa_private_key_size

slhdsa_private_key_size() -> int

Description: Return the SLH-DSA private-key size.

slhdsa_signature_size

slhdsa_signature_size() -> int

Description: Return the SLH-DSA detached signature size.

slhdsa_signed_message_size

slhdsa_signed_message_size(message_length: int) -> int

Description: Return the SLH-DSA signed-message size for a message length.

slhdsa_generate_keypair

slhdsa_generate_keypair() -> PqcKeyPair

Description: Generate an SLH-DSA key pair.

try_slhdsa_generate_keypair

try_slhdsa_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate an SLH-DSA key pair into caller-owned buffers.

slhdsa_sign

slhdsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Sign a message with SLH-DSA and return QSCABI signed-message bytes.

try_slhdsa_sign

try_slhdsa_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, signed_message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Sign an SLH-DSA message into caller-owned storage.

slhdsa_verify

slhdsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Verify an SLH-DSA signed message and recover the original message.

try_slhdsa_verify

try_slhdsa_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Verify an SLH-DSA signed message into caller-owned storage.

falcon_public_key_size

falcon_public_key_size() -> int

Description: Return the Falcon public-key size.

falcon_private_key_size

falcon_private_key_size() -> int

Description: Return the Falcon private-key size.

falcon_signature_size

falcon_signature_size() -> int

Description: Return the Falcon detached signature size.

falcon_signed_message_size

falcon_signed_message_size(message_length: int) -> int

Description: Return the Falcon signed-message size for a message length.

falcon_generate_keypair

falcon_generate_keypair() -> PqcKeyPair

Description: Generate a Falcon key pair.

try_falcon_generate_keypair

try_falcon_generate_keypair(public_key: bytearray | memoryview, private_key: bytearray | memoryview) -> QscStatus

Description: Generate a Falcon key pair into caller-owned buffers.

falcon_sign

falcon_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview) -> bytes

Description: Sign a message with Falcon and return QSCABI signed-message bytes.

try_falcon_sign

try_falcon_sign(message: bytes | bytearray | memoryview, private_key: bytes | bytearray | memoryview, signed_message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Sign a Falcon message into caller-owned storage.

falcon_verify

falcon_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview) -> bytes

Description: Verify a Falcon signed message and recover the original message.

try_falcon_verify

try_falcon_verify(signed_message: bytes | bytearray | memoryview, public_key: bytes | bytearray | memoryview, message: bytearray | memoryview) -> tuple[QscStatus, int]

Description: Verify a Falcon signed message into caller-owned storage.

24.18 qsc.status (2 Python entries)

Implementation source: qsc/status.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

status_string

status_string(status: int | QscStatus) -> str

Description: Return the native diagnostic label for a status code.

is_success

is_success(status: int | QscStatus) -> bool

Description: Return True when a status value represents successful completion.

24.19 qsc.stream (10 Python entries)

Implementation source: qsc/stream.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

available

available() -> bool

Description: Report whether the loaded DLL supports incremental digest creation.

Digest.update

update(self, data: bytes | bytearray | memoryview) -> None

Description: Absorb borrowed message bytes; empty updates are permitted.

Digest.finalize

finalize(self) -> bytes

Description: Return exact digest byte count, refusing repeated finalization.

Xof.absorb

absorb(self, data: bytes | bytearray | memoryview) -> None

Description: Absorb message bytes prior to finalization, including empty input.

Xof.finalize

finalize(self) -> None

Description: Transition the SHAKE sponge from absorption to squeezing once.

Xof.squeeze

squeeze(self, length: int) -> bytes

Description: Return the next output bytes from the finalized sponge state.

cshake

cshake(algorithm: XofAlgorithm, input_data: bytes | bytearray | memoryview, output_length: int, *, name: bytes=b'', custom: bytes=b'') -> bytes

Description: Compute customized cSHAKE with exact function name and customization.

Notes: This unkeyed transform does not authenticate the message or peer. The first input is the native `key` parameter, which carries input data.

Mac.update

update(self, data: bytes | bytearray | memoryview) -> None

Description: Absorb additional message bytes before finalizing.

Mac.finalize

finalize(self, tag_length: int) -> bytes

Description: Return exact HMAC tag or requested KMAC output length.

Notes: HMAC256/384/512 require 32/48/64 bytes respectively. KMAC allows nonzero caller-selected output length; verify tags before trusting data.

Ctr.transform

transform(self, data: bytes | bytearray | memoryview) -> bytes

Description: Transform message bytes; reject operations after a partial block.

Notes: CTR provides confidentiality only, no authentication or tamper check.

24.20 qsc.symmetric (39 Python entries)

Implementation source: qsc/symmetric.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

aes_block_size

aes_block_size() -> int

Description: Return the AES block and AES-CTR counter-block size.

aes128_key_size

aes128_key_size() -> int

Description: Return the AES-128 key length.

aes256_key_size

aes256_key_size() -> int

Description: Return the AES-256 key length.

aes_gcm_nonce_size

aes_gcm_nonce_size() -> int

Description: Return the preferred AES-GCM nonce length.

aes_gcm_max_nonce_size

aes_gcm_max_nonce_size() -> int

Description: Return the maximum AES-GCM nonce length accepted by QSCABI.

aes_gcm_tag_size

aes_gcm_tag_size() -> int

Description: Return the AES-GCM authentication-tag length.

chacha20_key_size

chacha20_key_size() -> int

Description: Return the ChaCha20 key length.

chacha20_nonce_size

chacha20_nonce_size() -> int

Description: Return the ChaCha20 nonce length.

chacha20_poly1305_tag_size

chacha20_poly1305_tag_size() -> int

Description: Return the ChaCha20-Poly1305 authentication-tag length.

csx_key_size

csx_key_size() -> int

Description: Return the CSX key length.

csx_nonce_size

csx_nonce_size() -> int

Description: Return the CSX nonce length.

csx_tag_size

csx_tag_size() -> int

Description: Return the CSX authentication-tag length.

rcs_key_size

rcs_key_size(mode: QscRcsMode | int) -> int

Description: Return the RCS key length for the selected profile.

rcs_nonce_size

rcs_nonce_size() -> int

Description: Return the RCS nonce length.

rcs_tag_size

rcs_tag_size(mode: QscRcsMode | int) -> int

Description: Return the RCS authentication-tag length for the selected profile.

aes256_ctr_transform

aes256_ctr_transform(data: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, counter_mode: QscAesCtrCounter | int=QscAesCtrCounter.BIG_ENDIAN) -> bytes

Description: Transform data using AES-256-CTR.

try_aes256_ctr_transform

try_aes256_ctr_transform(output: bytearray | memoryview, data: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, counter_mode: QscAesCtrCounter | int=QscAesCtrCounter.BIG_ENDIAN) -> QscStatus

Description: Transform AES-256-CTR data into caller-owned storage.

chacha20_transform

chacha20_transform(data: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview) -> bytes

Description: Transform data using ChaCha20.

try_chacha20_transform

try_chacha20_transform(output: bytearray | memoryview, data: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview) -> QscStatus

Description: Transform ChaCha20 data into caller-owned storage.

aes128_gcm_encrypt

aes128_gcm_encrypt(plaintext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Encrypt with AES-128-GCM and return ciphertext with the tag appended.

aes128_gcm_decrypt

aes128_gcm_decrypt(ciphertext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Decrypt and authenticate AES-128-GCM ciphertext.

aes256_gcm_encrypt

aes256_gcm_encrypt(plaintext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Encrypt with AES-256-GCM and return ciphertext with the tag appended.

aes256_gcm_decrypt

aes256_gcm_decrypt(ciphertext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Decrypt and authenticate AES-256-GCM ciphertext.

try_aes128_gcm_encrypt

try_aes128_gcm_encrypt(output, plaintext, key, nonce, aad=b'') -> tuple[QscStatus, int]

Description: Encrypt AES-128-GCM into caller-owned storage.

try_aes128_gcm_decrypt

try_aes128_gcm_decrypt(output, ciphertext, key, nonce, aad=b'') -> tuple[QscStatus, int]

Description: Decrypt AES-128-GCM into caller-owned storage.

try_aes256_gcm_encrypt

try_aes256_gcm_encrypt(output, plaintext, key, nonce, aad=b'') -> tuple[QscStatus, int]

Description: Encrypt AES-256-GCM into caller-owned storage.

try_aes256_gcm_decrypt

try_aes256_gcm_decrypt(output, ciphertext, key, nonce, aad=b'') -> tuple[QscStatus, int]

Description: Decrypt AES-256-GCM into caller-owned storage.

chacha20_poly1305_encrypt

chacha20_poly1305_encrypt(plaintext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Encrypt with ChaCha20-Poly1305 and return ciphertext with the tag appended.

chacha20_poly1305_decrypt

chacha20_poly1305_decrypt(ciphertext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Decrypt and authenticate ChaCha20-Poly1305 ciphertext.

try_chacha20_poly1305_encrypt

try_chacha20_poly1305_encrypt(output, plaintext, key, nonce, aad=b'') -> tuple[QscStatus, int]

Description: Encrypt ChaCha20-Poly1305 into caller-owned storage.

try_chacha20_poly1305_decrypt

try_chacha20_poly1305_decrypt(output, ciphertext, key, nonce, aad=b'') -> tuple[QscStatus, int]

Description: Decrypt ChaCha20-Poly1305 into caller-owned storage.

csx_encrypt

csx_encrypt(plaintext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, info: bytes | bytearray | memoryview=b'', aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Encrypt with CSX and return ciphertext with the tag appended.

csx_decrypt

csx_decrypt(ciphertext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, info: bytes | bytearray | memoryview=b'', aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Decrypt and authenticate CSX ciphertext.

try_csx_encrypt

try_csx_encrypt(output, plaintext, key, nonce, info=b'', aad=b'') -> tuple[QscStatus, int]

Description: Encrypt CSX into caller-owned storage.

try_csx_decrypt

try_csx_decrypt(output, ciphertext, key, nonce, info=b'', aad=b'') -> tuple[QscStatus, int]

Description: Decrypt CSX into caller-owned storage.

rcs_encrypt

rcs_encrypt(mode: QscRcsMode | int, plaintext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, info: bytes | bytearray | memoryview=b'', aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Encrypt with the selected RCS AEAD profile.

rcs_decrypt

rcs_decrypt(mode: QscRcsMode | int, ciphertext: bytes | bytearray | memoryview, key: bytes | bytearray | memoryview, nonce: bytes | bytearray | memoryview, info: bytes | bytearray | memoryview=b'', aad: bytes | bytearray | memoryview=b'') -> bytes

Description: Decrypt and authenticate ciphertext with the selected RCS AEAD profile.

try_rcs_encrypt

try_rcs_encrypt(mode, output, plaintext, key, nonce, info=b'', aad=b'') -> tuple[QscStatus, int]

Description: Encrypt RCS into caller-owned storage.

try_rcs_decrypt

try_rcs_decrypt(mode, output, ciphertext, key, nonce, info=b'', aad=b'') -> tuple[QscStatus, int]

Description: Decrypt RCS into caller-owned storage.

24.21 qsc.tls (87 Python entries)

Implementation source: qsc/tls.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

tls_status_string

tls_status_string(status: int | QscStatus) -> str

Description: Return the native TLS diagnostic string for a QSCABI status code.

TlsOptions.default

default(cls) -> 'TlsOptions'

Description: Return the native default TLS socket options.

TlsOptions.from_native

from_native(cls, options: TlsSocketOptions) -> 'TlsOptions'

Description: Convert a native TLS socket-options structure to a Python object.

TlsOptions.to_native

to_native(self) -> TlsSocketOptions

Description: Convert this object to a native TLS socket-options structure.

TlsSessionTicketPolicy.default

default(cls) -> 'TlsSessionTicketPolicy'

Description: Return the native default TLS session-ticket policy.

TlsSessionTicketPolicy.from_native

from_native(cls, policy: TlsTicketPolicy) -> 'TlsSessionTicketPolicy'

Description: Convert a native TLS ticket-policy structure to a Python object.

TlsSessionTicketPolicy.to_native

to_native(self) -> TlsTicketPolicy

Description: Convert this object to a native TLS ticket-policy structure.

TlsPeerInformation.from_native

from_native(cls, info: TlsPeerInfo) -> 'TlsPeerInformation'

Description: Convert a native peer-info structure to an immutable Python snapshot.

TlsContext.create

create(cls) -> 'TlsContext'

Description: Create a TLS context and raise QscError on native failure.

TlsContext.try_create

try_create(cls) -> tuple[QscStatus, 'TlsContext' | None]

Description: Attempt to create a TLS context without throwing on QSCABI failure.

TlsContext.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper calls.

TlsContext.set_policy

set_policy(self, policy: QscTlsPolicy | int) -> None

Description: Apply a predefined TLS policy to this context.

TlsContext.try_set_policy

try_set_policy(self, policy: QscTlsPolicy | int) -> QscStatus

Description: Attempt to apply a predefined TLS policy.

TlsContext.set_socket_options

set_socket_options(self, options: TlsOptions) -> None

Description: Apply socket options to this context.

TlsContext.try_set_socket_options

try_set_socket_options(self, options: TlsOptions) -> QscStatus

Description: Attempt to apply socket options to this context.

TlsContext.set_alpn_protocols

set_alpn_protocols(self, protocols: Sequence[str | bytes], required: bool=False) -> None

Description: Set the ordered ALPN protocol list for this context.

TlsContext.try_set_alpn_protocols

try_set_alpn_protocols(self, protocols: Sequence[str | bytes], required: bool=False) -> QscStatus

Description: Attempt to set the ordered ALPN protocol list for this context.

TlsContext.clear_alpn_protocols

clear_alpn_protocols(self) -> None

Description: Clear the ALPN protocol list from this context.

TlsContext.try_clear_alpn_protocols

try_clear_alpn_protocols(self) -> QscStatus

Description: Attempt to clear the ALPN protocol list from this context.

TlsContext.set_sni_required

set_sni_required(self, required: bool) -> None

Description: Configure whether SNI is required for the context.

TlsContext.try_set_sni_required

try_set_sni_required(self, required: bool) -> QscStatus

Description: Attempt to configure whether SNI is required for the context.

TlsContext.set_client_auth

set_client_auth(self, request_client_auth: bool, require_client_auth: bool) -> None

Description: Configure client-certificate authentication policy.

TlsContext.try_set_client_auth

try_set_client_auth(self, request_client_auth: bool, require_client_auth: bool) -> QscStatus

Description: Attempt to configure client-certificate authentication policy.

TlsContext.set_ticket_policy

set_ticket_policy(self, policy: TlsSessionTicketPolicy) -> None

Description: Apply TLS session-ticket policy to this context.

TlsContext.try_set_ticket_policy

try_set_ticket_policy(self, policy: TlsSessionTicketPolicy) -> QscStatus

Description: Attempt to apply TLS session-ticket policy to this context.

TlsContext.load_trust_anchor_pem

load_trust_anchor_pem(self, pem: str | bytes | bytearray | memoryview, self_signed: bool=True) -> None

Description: Load a PEM trust anchor into this TLS context.

TlsContext.try_load_trust_anchor_pem

try_load_trust_anchor_pem(self, pem: str | bytes | bytearray | memoryview, self_signed: bool=True) -> QscStatus

Description: Attempt to load a PEM trust anchor into this TLS context.

TlsContext.load_server_identity_pem

load_server_identity_pem(self, chain_pem: str | bytes | bytearray | memoryview, private_key_pem: str | bytes | bytearray | memoryview, signature_scheme: QscTlsSignatureScheme | int) -> None

Description: Load PEM server certificate-chain and private-key identity material.

TlsContext.try_load_server_identity_pem

try_load_server_identity_pem(self, chain_pem: str | bytes | bytearray | memoryview, private_key_pem: str | bytes | bytearray | memoryview, signature_scheme: QscTlsSignatureScheme | int) -> QscStatus

Description: Attempt to load PEM server certificate-chain and private-key identity material.

TlsContext.set_cipher_suites

set_cipher_suites(self, suites) -> None

Description: Set ordered TLS 1.3 cipher suite wire IDs, e.g. 0x1301/0x1302.

Notes: The native library rejects unsupported identifiers. This does not authenticate any peer or infer whether a cipher will be negotiated.

TlsContext.set_named_groups

set_named_groups(self, groups) -> None

Description: Set preferred TLS named group wire IDs, e.g. secp256r1 0x0017.

TlsContext.set_signature_ids

set_signature_ids(self, schemes) -> None

Description: Set TLS CertificateVerify signature wire IDs, e.g. ECDSA 0x0403.

TlsContext.load_trust_anchor_file

load_trust_anchor_file(self, path, *, self_signed: bool=True) -> None

Description: Load an authorized PEM trust-anchor file; caller controls file provenance.

TlsContext.load_trust_anchor_bundle_file

load_trust_anchor_bundle_file(self, path, *, self_signed: bool=True) -> None

Description: Load a PEM trust-anchor bundle using native certificate validation.

TlsContext.load_crl_file

load_crl_file(self, path) -> None

Description: Load a CRL file into the native TLS context's revocation policy.

TlsContext.load_server_identity_files

load_server_identity_files(self, chain_path, key_path, scheme: QscTlsSignatureScheme | int) -> None

Description: Load server certificate chain and private key files under native custody.

TlsContext.load_client_identity_files

load_client_identity_files(self, chain_path, key_path, scheme: QscTlsSignatureScheme | int) -> None

Description: Load client certificate chain and private key files for mutual TLS.

TlsContext.add_server_identity_files

add_server_identity_files(self, hostname: str, chain_path, key_path, scheme: QscTlsSignatureScheme | int) -> None

Description: Configure a hostname-specific server chain and private key (SNI).

Notes: Hostname must be caller-validated under DNS/IDNA policy; this method rejects empty or embedded-NUL names and does not authenticate a peer.

TlsContext.close

close(self) -> None

Description: Release the native context handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

TlsConnection.create

create(cls) -> 'TlsConnection'

Description: Create an idle TLS connection handle.

TlsConnection.try_create

try_create(cls) -> tuple[QscStatus, 'TlsConnection' | None]

Description: Attempt to create an idle TLS connection handle.

TlsConnection.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper calls.

TlsConnection.is_connected [property]

is_connected(self) -> bool

Description: Return True when the native connection is connected.

TlsConnection.is_handshake_complete [property]

is_handshake_complete(self) -> bool

Description: Return True when the native connection handshake is complete.

TlsConnection.connect_host

connect_host(self, context: TlsContext, hostname: str, service: str) -> None

Description: Connect this TLS client connection to a host and service.

TlsConnection.try_connect_host

try_connect_host(self, context: TlsContext, hostname: str, service: str) -> QscStatus

Description: Attempt to connect this TLS client connection to a host and service.

TlsConnection.send

send(self, data: bytes | bytearray | memoryview) -> int

Description: Send application data and return the number of bytes accepted by QSCABI.

TlsConnection.try_send

try_send(self, data: bytes | bytearray | memoryview) -> tuple[QscStatus, int]

Description: Attempt to send application data.

TlsConnection.receive

receive(self, output_length: int) -> bytes

Description: Receive application data into a newly allocated bytes object.

TlsConnection.try_receive

try_receive(self, output_length: int) -> tuple[QscStatus, bytes]

Description: Attempt to receive application data.

TlsConnection.send_frame

send_frame(self, data: bytes | bytearray | memoryview) -> None

Description: Send one length-delimited TLS frame.

TlsConnection.try_send_frame

try_send_frame(self, data: bytes | bytearray | memoryview) -> QscStatus

Description: Attempt to send one length-delimited TLS frame.

TlsConnection.receive_frame

receive_frame(self, output_length: int) -> bytes

Description: Receive one length-delimited TLS frame.

TlsConnection.try_receive_frame

try_receive_frame(self, output_length: int) -> tuple[QscStatus, bytes]

Description: Attempt to receive one length-delimited TLS frame.

TlsConnection.shutdown

shutdown(self) -> None

Description: Perform an orderly TLS shutdown.

TlsConnection.try_shutdown

try_shutdown(self) -> QscStatus

Description: Attempt an orderly TLS shutdown.

TlsConnection.cancel

cancel(self) -> None

Description: Cancel this TLS connection.

TlsConnection.try_cancel

try_cancel(self) -> QscStatus

Description: Attempt to cancel this TLS connection.

TlsConnection.selected_alpn

selected_alpn(self, initial_size: int=DEFAULT_ALPN_BUFFER_SIZE) -> str

Description: Return the selected ALPN value reported by the native connection.

TlsConnection.try_selected_alpn

try_selected_alpn(self, initial_size: int=DEFAULT_ALPN_BUFFER_SIZE) -> tuple[QscStatus, str]

Description: Attempt to read the selected ALPN value.

TlsConnection.peer_info

peer_info(self) -> TlsPeerInformation

Description: Return a peer-information snapshot.

TlsConnection.try_peer_info

try_peer_info(self) -> tuple[QscStatus, TlsPeerInformation | None]

Description: Attempt to return a peer-information snapshot.

TlsConnection.close

close(self) -> None

Description: Release the native connection handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

TlsListener.create

create(cls) -> 'TlsListener'

Description: Create a TLS listener handle.

TlsListener.try_create

try_create(cls) -> tuple[QscStatus, 'TlsListener' | None]

Description: Attempt to create a TLS listener handle.

TlsListener.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper calls.

TlsListener.set_socket_options

set_socket_options(self, options: TlsOptions) -> None

Description: Apply socket options to this listener.

TlsListener.try_set_socket_options

try_set_socket_options(self, options: TlsOptions) -> QscStatus

Description: Attempt to apply socket options to this listener.

TlsListener.bind

bind(self, context: TlsContext, address: str, port: int, family: QscTlsAddressFamily | int) -> None

Description: Bind the listener to a local address and port.

TlsListener.try_bind

try_bind(self, context: TlsContext, address: str, port: int, family: QscTlsAddressFamily | int) -> QscStatus

Description: Attempt to bind the listener to a local address and port.

TlsListener.accept

accept(self, connection: TlsConnection | None=None) -> TlsConnection

Description: Accept a TLS connection into a supplied or newly created connection handle.

TlsListener.try_accept

try_accept(self, connection: TlsConnection | None=None) -> tuple[QscStatus, TlsConnection | None]

Description: Attempt to accept a TLS connection.

TlsListener.close_listener

close_listener(self) -> None

Description: Close the underlying listener endpoint without releasing the handle.

TlsListener.close

close(self) -> None

Description: Release the native listener handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

TlsServer.create

create(cls) -> 'TlsServer'

Description: Create a TLS server handle.

TlsServer.try_create

try_create(cls) -> tuple[QscStatus, 'TlsServer' | None]

Description: Attempt to create a TLS server handle.

TlsServer.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper calls.

TlsServer.configure

configure(self, context: TlsContext, address: str, port: int, family: QscTlsAddressFamily | int) -> None

Description: Configure the server endpoint.

TlsServer.try_configure

try_configure(self, context: TlsContext, address: str, port: int, family: QscTlsAddressFamily | int) -> QscStatus

Description: Attempt to configure the server endpoint.

TlsServer.set_max_clients

set_max_clients(self, max_clients: int) -> None

Description: Set the maximum number of native server clients.

TlsServer.try_set_max_clients

try_set_max_clients(self, max_clients: int) -> QscStatus

Description: Attempt to set the maximum number of native server clients.

TlsServer.start

start(self) -> None

Description: Start the configured server in native single-server mode.

TlsServer.try_start

try_start(self) -> QscStatus

Description: Attempt to start the configured server in native single-server mode.

TlsServer.start_concurrent

start_concurrent(self) -> None

Description: Start the configured server in native concurrent mode.

TlsServer.try_start_concurrent

try_start_concurrent(self) -> QscStatus

Description: Attempt to start the configured server in native concurrent mode.

TlsServer.stop

stop(self) -> None

Description: Stop the native server endpoint.

TlsServer.close

close(self) -> None

Description: Stop and release the native server handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

24.22 qsc.version (9 Python entries)

Implementation source: qsc/version.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

major

major() -> int

Description: Major through the QSCPY QSCABI and linked QSC version/build metadata interface.

minor

minor() -> int

Description: Minor through the QSCPY QSCABI and linked QSC version/build metadata interface.

patch

patch() -> int

Description: Patch through the QSCPY QSCABI and linked QSC version/build metadata interface.

abi_version

abi_version() -> str

Description: Abi version through the QSCPY QSCABI and linked QSC version/build metadata interface.

qsc_version

qsc_version() -> str

Description: Qsc version through the QSCPY QSCABI and linked QSC version/build metadata interface.

build_platform

build_platform() -> str

Description: Build platform through the QSCPY QSCABI and linked QSC version/build metadata interface.

build_compiler

build_compiler() -> str

Description: Build compiler through the QSCPY QSCABI and linked QSC version/build metadata interface.

enabled_features

enabled_features() -> str

Description: Enabled features through the QSCPY QSCABI and linked QSC version/build metadata interface.

get_version_info

get_version_info() -> VersionInfo

Description: Return all version and build fields in one immutable object.

24.23 qsc.x509 (42 Python entries)

Implementation source: qsc/x509.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

x509_status_string

x509_status_string(status: int | QscStatus) -> str

Description: Return the native X.509 diagnostic string for a QSCABI status code.

X509Certificate.load_memory

load_memory(cls, data: bytes | bytearray | memoryview) -> 'X509Certificate'

Description: Load a certificate from DER, PEM, or another memory encoding accepted by QSCABI.

X509Certificate.try_load_memory

try_load_memory(cls, data: bytes | bytearray | memoryview) -> tuple[QscStatus, 'X509Certificate' | None]

Description: Attempt to load a certificate from a caller-owned memory buffer.

X509Certificate.load_pem

load_pem(cls, pem: str | bytes | bytearray | memoryview) -> 'X509Certificate'

Description: Load a certificate from PEM text.

X509Certificate.try_load_pem

try_load_pem(cls, pem: str | bytes | bytearray | memoryview) -> tuple[QscStatus, 'X509Certificate' | None]

Description: Attempt to load a certificate from PEM text.

X509Certificate.load_der

load_der(cls, der: bytes | bytearray | memoryview) -> 'X509Certificate'

Description: Load a certificate from DER bytes.

X509Certificate.try_load_der

try_load_der(cls, der: bytes | bytearray | memoryview) -> tuple[QscStatus, 'X509Certificate' | None]

Description: Attempt to load a certificate from DER bytes.

X509Certificate.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper use.

X509Certificate.is_closed [property]

is_closed(self) -> bool

Description: Return True after this object has released its native handle.

X509Certificate.subject [property]

subject(self) -> str

Description: Return the certificate subject distinguished-name string.

X509Certificate.issuer [property]

issuer(self) -> str

Description: Return the certificate issuer distinguished-name string.

X509Certificate.subject_common_name [property]

subject_common_name(self) -> str

Description: Return the certificate subject common-name value.

X509Certificate.dns_name_count [property]

dns_name_count(self) -> int

Description: Return the number of DNS subject-alternative-name entries.

X509Certificate.get_dns_name

get_dns_name(self, index: int) -> str

Description: Return one DNS subject-alternative-name entry by zero-based index.

X509Certificate.export_der

export_der(self) -> bytes

Description: Export this certificate as DER bytes.

X509Certificate.export_pem

export_pem(self) -> str

Description: Export this certificate as PEM text.

X509Certificate.try_match_hostname

try_match_hostname(self, hostname: str) -> QscStatus

Description: Return the QSCABI hostname-match status for this certificate.

X509Certificate.matches_hostname

matches_hostname(self, hostname: str) -> bool

Description: Return True when this certificate matches the supplied hostname.

X509Certificate.try_check_role

try_check_role(self, role: QscX509CertificateRole | int, hostname: str | None=None) -> QscStatus

Description: Return the QSCABI role-check status for this certificate.

X509Certificate.check_role

check_role(self, role: QscX509CertificateRole | int, hostname: str | None=None) -> bool

Description: Return True when this certificate satisfies the requested role.

X509Certificate.close

close(self) -> None

Description: Release the native certificate handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

X509Chain.create

create(cls, certificate_capacity: int) -> 'X509Chain'

Description: Create an empty certificate chain with the requested certificate capacity.

X509Chain.try_create

try_create(cls, certificate_capacity: int) -> tuple[QscStatus, 'X509Chain' | None]

Description: Attempt to create an empty certificate chain.

X509Chain.load_memory

load_memory(cls, data: bytes | bytearray | memoryview, certificate_capacity: int) -> 'X509Chain'

Description: Load a certificate chain from memory.

X509Chain.try_load_memory

try_load_memory(cls, data: bytes | bytearray | memoryview, certificate_capacity: int) -> tuple[QscStatus, 'X509Chain' | None]

Description: Attempt to load a certificate chain from memory.

X509Chain.load_pem

load_pem(cls, pem: str | bytes | bytearray | memoryview, certificate_capacity: int) -> 'X509Chain'

Description: Load a certificate chain from concatenated PEM text.

X509Chain.try_load_pem

try_load_pem(cls, pem: str | bytes | bytearray | memoryview, certificate_capacity: int) -> tuple[QscStatus, 'X509Chain' | None]

Description: Attempt to load a certificate chain from concatenated PEM text.

X509Chain.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper use.

X509Chain.count [property]

count(self) -> int

Description: Return the number of certificates loaded in this chain.

X509Chain.close

close(self) -> None

Description: Release the native chain handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

X509TrustStore.create

create(cls, anchor_capacity: int) -> 'X509TrustStore'

Description: Create an empty trust store with the requested trust-anchor capacity.

X509TrustStore.try_create

try_create(cls, anchor_capacity: int) -> tuple[QscStatus, 'X509TrustStore' | None]

Description: Attempt to create an empty trust store.

X509TrustStore.handle [property]

handle(self) -> Handle

Description: Return the raw native handle for internal wrapper use.

X509TrustStore.count [property]

count(self) -> int

Description: Return the number of trust anchors in this store.

X509TrustStore.try_add_anchor_memory

try_add_anchor_memory(self, data: bytes | bytearray | memoryview, self_signed: bool) -> QscStatus

Description: Return the QSCABI status after adding a trust anchor from memory.

X509TrustStore.add_anchor_memory

add_anchor_memory(self, data: bytes | bytearray | memoryview, self_signed: bool) -> None

Description: Add a trust anchor from memory, raising QscError on failure.

X509TrustStore.try_add_anchor_pem

try_add_anchor_pem(self, pem: str | bytes | bytearray | memoryview, self_signed: bool) -> QscStatus

Description: Return the QSCABI status after adding a PEM trust anchor.

X509TrustStore.add_anchor_pem

add_anchor_pem(self, pem: str | bytes | bytearray | memoryview, self_signed: bool) -> None

Description: Add a PEM trust anchor, raising QscError on failure.

X509TrustStore.try_verify_peer_certificates

try_verify_peer_certificates(self, chain: X509Chain, preset: QscX509ProfilePreset | int, hostname: str | None=None) -> QscStatus

Description: Return the QSCABI peer-verification status for a chain and optional hostname.

X509TrustStore.verify_peer_certificates

verify_peer_certificates(self, chain: X509Chain, preset: QscX509ProfilePreset | int, hostname: str | None=None) -> bool

Description: Return True when the peer chain validates against this trust store.

X509TrustStore.add_crl_memory

add_crl_memory(self, der: bytes | bytearray | memoryview) -> None

Description: Load a signed DER CRL into this existing native trust store.

Notes: The DER bytes are borrowed synchronously. Adding a CRL alone is not complete revocation enforcement; native verification policy controls whether stored revocation evidence is applied to peer certificates.

X509TrustStore.close

close(self) -> None

Description: Release the native trust-store handle.

Notes: Returns: Completes a lifecycle or clear operation (declared return: None); do not use the closed handle or cleared output again.

24.24 qsc.xof (6 Python entries)

Implementation source: qsc/xof.py. All signatures below belong to this module. Native arguments must conform to their underlying ctypes ABI declarations; see Section 25.

shake_128

shake_128(message: bytes | bytearray | memoryview, output_length: int) -> bytes

Description: Compute SHAKE-128 output of the requested length.

shake_256

shake_256(message: bytes | bytearray | memoryview, output_length: int) -> bytes

Description: Compute SHAKE-256 output of the requested length.

shake_512

shake_512(message: bytes | bytearray | memoryview, output_length: int) -> bytes

Description: Compute SHAKE-512 output of the requested length.

try_shake_128

try_shake_128(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHAKE-128 into caller-owned storage.

try_shake_256

try_shake_256(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHAKE-256 into caller-owned storage.

try_shake_512

try_shake_512(output: bytearray | memoryview, message: bytes | bytearray | memoryview) -> QscStatus

Description: Compute SHAKE-512 into caller-owned storage.

25: Native ctypes ABI Function Registry

This appendix preserves the exact 329-name qscabi_* function registry from qsc/_native/bindings.py. The entries list the C symbol, the wrapper's native module dispatch key, the ctypes result type, and the argument type tuple. The registry is a declaration map rather than proof that every symbol exists in the native binary currently loaded. Compare it to qsc/abi010.py EXPECTED_EXPORTS and the authoritative native QSCABI export manifest during release qualification.

25.1 abi010_native — 4 native symbols

Export name	Result type	ctypes argument types
qscabi_abi_is_compatible	c_uint32	(c_uint32, c_uint32)
qscabi_build_backend	c_char_p	()
qscabi_capability_flags	c_uint64	()
qscabi_qsc_build_profile	c_char_p	()

Table 25.1 Registered native declarations: abi010_native

25.2 asymmetric_native — 36 native symbols

Export name	Result type	ctypes argument types
qscabi_ecdh_public_key_size	c_size_t	()
qscabi_ecdh_private_key_size	c_size_t	()
qscabi_ecdh_shared_secret_size	c_size_t	()
qscabi_ecdh_seed_size	c_size_t	()
qscabi_ecdh_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_ecdh_generate_seeded_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_ecdh_public_from_private	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_ecdh_key_exchange	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddh_public_key_size	c_size_t	()
qscabi_eddh_private_key_size	c_size_t	()
qscabi_eddh_shared_secret_size	c_size_t	()
qscabi_eddh_seed_size	c_size_t	()
qscabi_eddh_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_eddh_generate_seeded_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddh_public_from_private	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddh_key_exchange	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_ecdsa_public_key_size	c_size_t	()
qscabi_ecdsa_private_key_size	c_size_t	()

Export name	Result type	ctypes argument types
qscabi_ecdsa_signature_size	c_size_t	()
qscabi_ecdsa_seed_size	c_size_t	()
qscabi_ecdsa_signed_message_size	c_size_t	(c_size_t,)
qscabi_ecdsa_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_ecdsa_generate_seeded_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_ecdsa_public_from_private	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_ecdsa_sign	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_ecdsa_verify	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddsa_public_key_size	c_size_t	()
qscabi_eddsa_private_key_size	c_size_t	()
qscabi_eddsa_signature_size	c_size_t	()
qscabi_eddsa_seed_size	c_size_t	()
qscabi_eddsa_signed_message_size	c_size_t	(c_size_t,)
qscabi_eddsa_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_eddsa_generate_seeded_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddsa_public_from_private	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddsa_sign	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_eddsa_verify	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.2 Registered native declarations: asymmetric_native

25.3 encoding_native — 12 native symbols

Export name	Result type	ctypes argument types
qscabi_hex_encoded_size	c_size_t	(c_size_t,)
qscabi_hex_decoded_size	c_int	(SizeTPtr, c_char_p, c_size_t,)
qscabi_hex_encode	c_int	(CharPtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t,)
qscabi_hex_decode	c_int	(BytePtr, c_size_t, SizeTPtr, c_char_p, c_size_t,)
qscabi_base64_encoded_size	c_size_t	(c_size_t,)
qscabi_base64_decoded_size	c_int	(SizeTPtr, c_char_p, c_size_t,)
qscabi_base64_encode	c_int	(CharPtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t,)
qscabi_base64_decode	c_int	(BytePtr, c_size_t, SizeTPtr, c_char_p, c_size_t,)
qscabi_pem_encoded_size	c_int	(SizeTPtr, c_char_p, c_size_t,)
qscabi_pem_decoded_size	c_int	(SizeTPtr, c_char_p, c_size_t,)
qscabi_pem_encode	c_int	(CharPtr, c_size_t, SizeTPtr, c_char_p, ConstBytePtr, c_size_t,)

Export name	Result type	ctypes argument types
qscabi_pem_decode	c_int	(BytePtr, c_size_t, SizeTPtr, c_char_p, c_size_t,)

Table 25.3 Registered native declarations: encoding_native

25.4 entropy_native — 10 native symbols

Export name	Result type	ctypes argument types
qscabi_entropy_acp_seed_max	c_size_t	()
qscabi_entropy_csp_seed_max	c_size_t	()
qscabi_entropy_rdp_seed_max	c_size_t	()
qscabi_entropy_cjp_seed_max	c_size_t	()
qscabi_entropy_rdp_available	c_int	()
qscabi_entropy_cjp_available	c_int	()
qscabi_entropy_acp_generate	c_int	(BytePtr, c_size_t,)
qscabi_entropy_csp_generate	c_int	(BytePtr, c_size_t,)
qscabi_entropy_rdp_generate	c_int	(BytePtr, c_size_t,)
qscabi_entropy_cjp_generate	c_int	(BytePtr, c_size_t,)

Table 25.4 Registered native declarations: entropy_native

25.5 explicit_curve_native — 5 native symbols

Export name	Result type	ctypes argument types
qscabi_curve_get_sizes	c_int	(c_int, CurveSizesPtr,)
qscabi_curve_generate	c_int	(c_int, BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_curve_generate_seeded	c_int	(c_int, BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_curve_public_from_private	c_int	(c_int, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_curve_agree	c_int	(c_int, BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.5 Registered native declarations: explicit_curve_native

25.6 explicit_kem_native — 4 native symbols

Export name	Result type	ctypes argument types
qscabi_kem_get_sizes	c_int	(c_int, KemSizesPtr,)
qscabi_kem_generate	c_int	(c_int, BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_kem_encapsulate	c_int	(c_int, BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kem_decapsulate	c_int	(c_int, BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.6 Registered native declarations: explicit_kem_native

25.7 explicit_signature_native — 4 native symbols

Export name	Result type	ctypes argument types
qscabi_signature_get_sizes	c_int	(c_int, SignatureSizesPtr,)
qscabi_signature_generate	c_int	(c_int, BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_signature_sign	c_int	(c_int, BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_signature_verify	c_int	(c_int, BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.7 Registered native declarations: explicit_signature_native

25.8 explicit_slhdsa_native — 4 native symbols

Export name	Result type	ctypes argument types
qscabi_slhdsa_get_sizes	c_int	(c_int, SlhDsaSizesPtr,)
qscabi_slhdsa_explicit_generate	c_int	(c_int, BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_slhdsa_explicit_sign	c_int	(c_int, c_int, c_bool, BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_slhdsa_explicit_verify	c_int	(c_int, c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.8 Registered native declarations: explicit_slhdsa_native

25.9 hardware_native — 14 native symbols

Export name	Result type	ctypes argument types
qscabi_hw_is_enabled	c_int	()
qscabi_hw_provider_open	c_int	(HandlePtr, HwBackendOpsPtr, Handle,)
qscabi_hw_provider_capabilities	c_int	(Handle, HwCapabilitiesPtr,)
qscabi_hw_key_open	c_int	(HandlePtr, Handle, c_char_p, HwPolicyPtr,)
qscabi_hw_key_generate	c_int	(HandlePtr, Handle, HwPolicyPtr,)
qscabi_hw_key_get_info	c_int	(Handle, HwKeyInfoPtr,)
qscabi_hw_key_public	c_int	(Handle, BytePtr, SizeTPtr,)
qscabi_hw_key_sign	c_int	(Handle, ConstBytePtr, c_size_t, BytePtr, SizeTPtr,)
qscabi_hw_key_agree	c_int	(Handle, ConstBytePtr, c_size_t, BytePtr, SizeTPtr,)
qscabi_hw_key_attest	c_int	(Handle, ConstBytePtr, c_size_t, BytePtr, SizeTPtr,)
qscabi_hw_provider_random	c_int	(Handle, BytePtr, c_size_t,)
qscabi_hw_provider_invalidate	c_int	(Handle,)
qscabi_hw_key_release	c_int	(HandlePtr,)
qscabi_hw_provider_close	c_int	(HandlePtr,)

Table 25.9 Registered native declarations: hardware_native

25.10 hash_native — 12 native symbols

Export name	Result type	ctypes argument types
qscabi_sha2_256_digest_size	c_size_t	()
qscabi_sha2_384_digest_size	c_size_t	()
qscabi_sha2_512_digest_size	c_size_t	()
qscabi_sha3_128_digest_size	c_size_t	()
qscabi_sha3_256_digest_size	c_size_t	()
qscabi_sha3_512_digest_size	c_size_t	()
qscabi_sha2_256_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_sha2_384_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_sha2_512_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_sha3_128_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_sha3_256_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_sha3_512_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.10 Registered native declarations: hash_native

25.11 kdf_native — 12 native symbols

Export name	Result type	ctypes argument types
qscabi_hkdf_sha2_256_prk_size	c_size_t	()
qscabi_hkdf_sha2_384_prk_size	c_size_t	()
qscabi_hkdf_sha2_512_prk_size	c_size_t	()
qscabi_pbmac1_max_key_size	c_size_t	()
qscabi_pbmac1_max_iterations	c_uint64	()
qscabi_hkdf_sha2_256_extract	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hkdf_sha2_256_expand	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hkdf_sha2_384_extract	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hkdf_sha2_384_expand	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hkdf_sha2_512_extract	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hkdf_sha2_512_expand	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_pbmac1_derive_key	c_int	(BytePtr, c_size_t, c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, c_uint64, c_size_t,)

Table 25.11 Registered native declarations: kdf_native

25.12 mac_native — 23 native symbols

Export name	Result type	ctypes argument types
qscabi_hmac_sha2_256_mac_size	c_size_t	()
qscabi_hmac_sha2_384_mac_size	c_size_t	()
qscabi_hmac_sha2_512_mac_size	c_size_t	()
qscabi_poly1305_mac_size	c_size_t	()
qscabi_poly1305_key_size	c_size_t	()
qscabi_qmac_mac_size	c_size_t	()
qscabi_qmac_key_size	c_size_t	()
qscabi_qmac_nonce_size	c_size_t	()
qscabi_pbmac1_mac_size	c_size_t	(c_int,)
qscabi_hmac_sha2_256_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hmac_sha2_384_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hmac_sha2_512_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kmac_128_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kmac_256_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kmac_512_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kmac_xof_128_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kmac_xof_256_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_kmac_xof_512_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_poly1305_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_poly1305_verify	c_int	(ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_qmac_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_pbmac1_compute	c_int	(BytePtr, c_size_t, c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, c_uint64, c_size_t, ConstBytePtr, c_size_t,)
qscabi_pbmac1_verify	c_int	(ConstBytePtr, c_size_t, c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, c_uint64, c_size_t, ConstBytePtr, c_size_t,)

Table 25.12 Registered native declarations: mac_native

25.13 memory_native — 2 native symbols

Export name	Result type	ctypes argument types
qscabi_memory_clear	None	(BytePtr, c_size_t,)
qscabi_memory_are_equal	c_int	(ConstBytePtr, ConstBytePtr, c_size_t,)

Table 25.13 Registered native declarations: memory_native

25.14 pqc_native — 42 native symbols

Export name	Result type	ctypes argument types
qscabi_mlkem_public_key_size	c_size_t	()
qscabi_mlkem_private_key_size	c_size_t	()
qscabi_mlkem_ciphertext_size	c_size_t	()
qscabi_mlkem_shared_secret_size	c_size_t	()
qscabi_mlkem_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_mlkem_encapsulate	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_mlkem_decapsulate	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hqc_public_key_size	c_size_t	()
qscabi_hqc_private_key_size	c_size_t	()
qscabi_hqc_ciphertext_size	c_size_t	()
qscabi_hqc_shared_secret_size	c_size_t	()
qscabi_hqc_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_hqc_encapsulate	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_hqc_decapsulate	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_mceliece_public_key_size	c_size_t	()
qscabi_mceliece_private_key_size	c_size_t	()
qscabi_mceliece_ciphertext_size	c_size_t	()
qscabi_mceliece_shared_secret_size	c_size_t	()
qscabi_mceliece_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_mceliece_encapsulate	c_int	(BytePtr, c_size_t, BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_mceliece_decapsulate	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_mldsa_public_key_size	c_size_t	()
qscabi_mldsa_private_key_size	c_size_t	()
qscabi_mldsa_signature_size	c_size_t	()
qscabi_mldsa_signed_message_size	c_size_t	(c_size_t,)
qscabi_mldsa_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_mldsa_sign	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Export name	Result type	ctypes argument types
qscabi_mldsa_verify	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_falcon_public_key_size	c_size_t	()
qscabi_falcon_private_key_size	c_size_t	()
qscabi_falcon_signature_size	c_size_t	()
qscabi_falcon_signed_message_size	c_size_t	(c_size_t,)
qscabi_falcon_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_falcon_sign	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_falcon_verify	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_slhdsa_public_key_size	c_size_t	()
qscabi_slhdsa_private_key_size	c_size_t	()
qscabi_slhdsa_signature_size	c_size_t	()
qscabi_slhdsa_signed_message_size	c_size_t	(c_size_t,)
qscabi_slhdsa_generate_keypair	c_int	(BytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_slhdsa_sign	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_slhdsa_verify	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.14 Registered native declarations: pqc_native

25.15 status_native — 1 native symbols

Export name	Result type	ctypes argument types
qscabi_status_string	c_char_p	(c_int,)

Table 25.15 Registered native declarations: status_native

25.16 stream_native — 18 native symbols

Export name	Result type	ctypes argument types
qscabi_digest_size	c_size_t	(c_int,)
qscabi_digest_create	c_int	(c_int, HandlePtr,)
qscabi_digest_update	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_digest_finalize	c_int	(Handle, BytePtr, c_size_t,)
qscabi_digest_destroy	None	(Handle,)
qscabi_xof_create	c_int	(c_int, HandlePtr,)
qscabi_xof_absorb	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_xof_finalize	c_int	(Handle,)
qscabi_xof_squeeze	c_int	(Handle, BytePtr, c_size_t,)
qscabi_xof_destroy	None	(Handle,)

Export name	Result type	ctypes argument types
qscabi_cshake_compute	c_int	(c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_stream_mac_create	c_int	(c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, HandlePtr,)
qscabi_stream_mac_update	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_stream_mac_finalize	c_int	(Handle, BytePtr, c_size_t,)
qscabi_stream_mac_destroy	None	(Handle,)
qscabi_ctr_create	c_int	(c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, HandlePtr,)
qscabi_ctr_transform	c_int	(Handle, ConstBytePtr, c_size_t, BytePtr, c_size_t,)
qscabi_ctr_destroy	None	(Handle,)

Table 25.16 Registered native declarations: stream_native

25.17 symmetric_native — 27 native symbols

Export name	Result type	ctypes argument types
qscabi_aes_block_size	c_size_t	()
qscabi_aes128_key_size	c_size_t	()
qscabi_aes256_key_size	c_size_t	()
qscabi_aes_gcm_nonce_size	c_size_t	()
qscabi_aes_gcm_max_nonce_size	c_size_t	()
qscabi_aes_gcm_tag_size	c_size_t	()
qscabi_chacha20_key_size	c_size_t	()
qscabi_chacha20_nonce_size	c_size_t	()
qscabi_chacha20_poly1305_tag_size	c_size_t	()
qscabi_csx_key_size	c_size_t	()
qscabi_csx_nonce_size	c_size_t	()
qscabi_csx_tag_size	c_size_t	()
qscabi_rcs_key_size	c_size_t	(c_int,)
qscabi_rcs_nonce_size	c_size_t	()
qscabi_rcs_tag_size	c_size_t	(c_int,)
qscabi_aes256_ctr_transform	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, c_int,)
qscabi_aes128_gcm_encrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_aes128_gcm_decrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Export name	Result type	ctypes argument types
qscabi_aes256_gcm_encrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_aes256_gcm_decrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_chacha20_transform	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_chacha20_poly1305_encrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_chacha20_poly1305_decrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_csx_encrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_csx_decrypt	c_int	(BytePtr, c_size_t, SizeTPtr, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_rcs_encrypt	c_int	(BytePtr, c_size_t, SizeTPtr, c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_rcs_decrypt	c_int	(BytePtr, c_size_t, SizeTPtr, c_int, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.17 Registered native declarations: symmetric_native

25.18 tls010_native — 9 native symbols

Export name	Result type	ctypes argument types
qscabi_tls_context_set_cipher_suites	c_int	(Handle, ctypes.POINTER(c_uint16), c_size_t)
qscabi_tls_context_set_named_groups	c_int	(Handle, ctypes.POINTER(c_uint16), c_size_t)
qscabi_tls_context_set_signature_ids	c_int	(Handle, ctypes.POINTER(c_uint16), c_size_t)
qscabi_tls_context_load_trust_anchor_file	c_int	(Handle, c_char_p, c_int)
qscabi_tls_context_load_trust_anchor_bundle_file	c_int	(Handle, c_char_p, c_int)
qscabi_tls_context_load_crl_file	c_int	(Handle, c_char_p)
qscabi_tls_context_load_server_identity_files	c_int	(Handle, c_char_p, c_char_p, c_int)
qscabi_tls_context_load_client_identity_files	c_int	(Handle, c_char_p, c_char_p, c_int)
qscabi_tls_context_add_server_identity_files	c_int	(Handle, c_char_p, c_char_p, c_char_p, c_int)

Table 25.18 Registered native declarations: tls010_native

25.19 tls_native — 40 native symbols

Export name	Result type	ctypes argument types
qscabi_tls_status_string	c_char_p	(c_int,)
qscabi_tls_socket_options_default	None	(TlsSocketOptionsPtr,)
qscabi_tls_ticket_policy_default	None	(TlsTicketPolicyPtr,)
qscabi_tls_context_create	c_int	(HandlePtr,)
qscabi_tls_context_destroy	None	(Handle,)
qscabi_tls_context_set_policy	c_int	(Handle, c_int,)
qscabi_tls_context_set_socket_options	c_int	(Handle, TlsSocketOptionsPtr,)
qscabi_tls_context_set_alpn_protocols	c_int	(Handle, CStringArrayPtr, c_size_t, c_int,)
qscabi_tls_context_clear_alpn_protocols	c_int	(Handle,)
qscabi_tls_context_set_sni_required	c_int	(Handle, c_int,)
qscabi_tls_context_set_client_auth	c_int	(Handle, c_int, c_int,)
qscabi_tls_context_set_ticket_policy	c_int	(Handle, TlsTicketPolicyPtr,)
qscabi_tls_context_load_trust_anchor_pem	c_int	(Handle, c_char_p, c_size_t, c_int,)
qscabi_tls_context_load_server_identity_pem	c_int	(Handle, c_char_p, c_size_t, c_char_p, c_size_t, c_int,)
qscabi_tls_connection_create	c_int	(HandlePtr,)
qscabi_tls_connection_destroy	None	(Handle,)
qscabi_tls_client_connect_host	c_int	(Handle, Handle, c_char_p, c_char_p,)
qscabi_tls_connection_send	c_int	(Handle, ConstBytePtr, c_size_t, SizeTPtr,)
qscabi_tls_connection_receive	c_int	(Handle, BytePtr, c_size_t, SizeTPtr,)
qscabi_tls_connection_send_frame	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_tls_connection_receive_frame	c_int	(Handle, BytePtr, c_size_t, SizeTPtr,)
qscabi_tls_connection_shutdown	c_int	(Handle,)
qscabi_tls_connection_cancel	c_int	(Handle,)
qscabi_tls_connection_is_connected	c_int	(Handle,)
qscabi_tls_connection_is_handshake_complete	c_int	(Handle,)
qscabi_tls_connection_get_selected_alpn	c_int	(Handle, CharPtr, c_size_t, SizeTPtr,)
qscabi_tls_connection_get_peer_info	c_int	(Handle, TlsPeerInfoPtr,)
qscabi_tls_listener_create	c_int	(HandlePtr,)
qscabi_tls_listener_destroy	None	(Handle,)
qscabi_tls_listener_set_socket_options	c_int	(Handle, TlsSocketOptionsPtr,)
qscabi_tls_listener_bind	c_int	(Handle, Handle, c_char_p, c_uint16, c_int,)
qscabi_tls_listener_accept	c_int	(Handle, Handle,)
qscabi_tls_listener_close	None	(Handle,)
qscabi_tls_server_create	c_int	(HandlePtr,)
qscabi_tls_server_destroy	None	(Handle,)
qscabi_tls_server_configure	c_int	(Handle, Handle, c_char_p, c_uint16, c_int,)
qscabi_tls_server_set_max_clients	c_int	(Handle, c_size_t,)
qscabi_tls_server_start	c_int	(Handle,)
qscabi_tls_server_start_concurrent	c_int	(Handle,)
qscabi_tls_server_stop	None	(Handle,)

Table 25.19 Registered native declarations: tls_native

25.20 version_native — 8 native symbols

Export name	Result type	ctypes argument types
qscabi_version_major	c_uint32	()
qscabi_version_minor	c_uint32	()
qscabi_version_patch	c_uint32	()
qscabi_version_string	c_char_p	()
qscabi_qsc_version_string	c_char_p	()
qscabi_build_platform	c_char_p	()
qscabi_build_compiler	c_char_p	()
qscabi_enabled_features	c_char_p	()

Table 25.20 Registered native declarations: version_native

25.21 x509010_native — 15 native symbols

Export name	Result type	ctypes argument types
qscabi_x509_csr_create	c_int	(HandlePtr,)
qscabi_x509_csr_destroy	None	(Handle,)
qscabi_x509_csr_init_p256	c_int	(Handle, c_char_p, ConstBytePtr, c_size_t)
qscabi_x509_csr_add_dns	c_int	(Handle, c_char_p)
qscabi_x509_csr_sign_der	c_int	(Handle, X509SignCallback, Handle, BytePtr, SizeTPtr)
qscabi_x509_csr_load	c_int	(Handle, ConstBytePtr, c_size_t)
qscabi_x509_csr_verify	c_int	(Handle,)
qscabi_x509_issue_tls_server_p256	c_int	(Handle, Handle, ConstBytePtr, c_size_t, X509UtcTimePtr, X509UtcTimePtr, X509SignCallback, Handle, BytePtr, SizeTPtr)
qscabi_x509_crl_builder_create	c_int	(HandlePtr, Handle, X509UtcTimePtr, X509UtcTimePtr)
qscabi_x509_crl_builder_destroy	None	(Handle,)
qscabi_x509_crl_set_number	c_int	(Handle, ConstBytePtr, c_size_t)
qscabi_x509_crl_add_revoked	c_int	(Handle, ConstBytePtr, c_size_t, X509UtcTimePtr)
qscabi_x509_crl_sign_der	c_int	(Handle, Handle, X509SignCallback, Handle, BytePtr, SizeTPtr)
qscabi_x509_crl_verify_der	c_int	(ConstBytePtr, c_size_t, Handle, X509UtcTimePtr)
qscabi_x509_store_add_crl_memory	c_int	(Handle, ConstBytePtr, c_size_t)

Table 25.21 Registered native declarations: x509010_native

25.22 x509_native — 24 native symbols

Export name	Result type	ctypes argument types
qscabi_x509_certificate_create	c_int	(HandlePtr,)
qscabi_x509_certificate_destroy	None	(Handle,)
qscabi_x509_certificate_load_memory	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_x509_certificate_load_pem	c_int	(Handle, c_char_p, c_size_t,)
qscabi_x509_certificate_load_der	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_x509_certificate_export_der	c_int	(Handle, BytePtr, c_size_t, SizeTPtr,)
qscabi_x509_certificate_export_pem	c_int	(Handle, CharPtr, c_size_t, SizeTPtr,)
qscabi_x509_certificate_subject_string	c_int	(Handle, CharPtr, c_size_t, SizeTPtr,)
qscabi_x509_certificate_issuer_string	c_int	(Handle, CharPtr, c_size_t, SizeTPtr,)
qscabi_x509_certificate_subject_common_name	c_int	(Handle, CharPtr, c_size_t, SizeTPtr,)
qscabi_x509_certificate_subject_dns_name_count	c_size_t	(Handle,)
qscabi_x509_certificate_subject_dns_name	c_int	(Handle, c_size_t, CharPtr, c_size_t, SizeTPtr,)
qscabi_x509_certificate_match_hostname	c_int	(Handle, c_char_p,)
qscabi_x509_certificate_check_role	c_int	(Handle, c_int, c_char_p,)
qscabi_x509_chain_create	c_int	(HandlePtr, c_size_t,)
qscabi_x509_chain_destroy	None	(Handle,)
qscabi_x509_chain_load_memory	c_int	(Handle, ConstBytePtr, c_size_t,)
qscabi_x509_chain_count	c_size_t	(Handle,)
qscabi_x509_store_create	c_int	(HandlePtr, c_size_t,)
qscabi_x509_store_destroy	None	(Handle,)
qscabi_x509_store_add_anchor_memory	c_int	(Handle, ConstBytePtr, c_size_t, c_int,)
qscabi_x509_store_count	c_size_t	(Handle,)
qscabi_x509_verify_peer_certificates	c_int	(Handle, Handle, c_int, c_char_p,)
qscabi_x509_status_string	c_char_p	(c_int,)

Table 25.22 Registered native declarations: x509_native

25.23 xof_native — 3 native symbols

Export name	Result type	ctypes argument types
qscabi_shake_128_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_shake_256_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)
qscabi_shake_512_compute	c_int	(BytePtr, c_size_t, ConstBytePtr, c_size_t,)

Table 25.23 Registered native declarations: xof_native

Annex A: Public Classes, Enums and Data Contracts

A.1 Public Python object families

Class	Public members	Category
abi010.NativeBuildInfo	0	dataclass
asymmetric.AsymmetricKeyPair	0	dataclass
errors.QscError	0	native/utility class
explicit_curve.CurveKeyPair	0	dataclass
explicit_kem.KemKeyPair	0	dataclass
explicit_kem.KemEncapsulation	0	dataclass
explicit_signature.SignatureKeyPair	0	dataclass
explicit_slhdsa.SlhDsaKeyPair	0	dataclass
issuance.P256DerSigner	1	native/utility class
issuance.X509Csr	8	native/utility class
issuance.X509CrlBuilder	5	native/utility class
pqc.PqcKeyPair	0	dataclass
pqc.KemEncapsulation	0	dataclass
status.QscStatus	0	IntEnum
stream.Digest	2	native/utility class
stream.Xof	3	native/utility class
stream.Mac	2	native/utility class
stream.Ctr	1	native/utility class
tls.TlsOptions	3	dataclass
tls.TlsSessionTicketPolicy	3	dataclass
tls.TlsPeerInformation	1	dataclass
tls.TlsContext	31	native/utility class
tls.TlsConnection	24	native/utility class
tls.TlsListener	11	native/utility class
tls.TlsServer	13	native/utility class
version.VersionInfo	0	dataclass
x509.X509Certificate	20	native/utility class
x509.X509Chain	9	native/utility class
x509.X509TrustStore	12	native/utility class

Table A.1 Public class catalog

A.2 Status values

Value	QscStatus member
0	SUCCESS
1	FAILURE
2	INVALID_ARGUMENT
3	INVALID_SIZE
4	BUFFER_TOO_SMALL
5	ALLOCATION_FAILURE

Value	QscStatus member
6	AUTHENTICATION_FAILURE
7	VERIFICATION_FAILURE
8	PLATFORM_UNAVAILABLE
9	UNSUPPORTED
10	NOT_INITIALIZED
11	ALREADY_INITIALIZED
12	INVALID_HANDLE
13	BAD_STATE
14	NOT_FOUND
15	IO_FAILURE
16	TLS_FAILURE
17	X509_FAILURE
18	CLOSED

Table A.2 Public native status identifiers

A.3 ctypes ABI primitives

Python alias	Native semantics / use
ctypes.c_size_t	Native-width byte counts; 64-bit in the supplied x64 DLL.
ctypes.c_uint8/16/32/64	Fixed-width C unsigned integer values and selector fields.
ctypes.c_int	Native signed C status return and enum-compatible parameter.
ctypes.c_void_p / Handle	Opaque QSCABI object pointer; lifetime governed by owner.
BytePtr / ConstBytePtr	Writable output vs borrowed input byte pointers.
SizeTPtr / HandlePtr	Out-parameters for length and allocated native state.
X509SignCallback	Synchronous DER-signing callback pointer; Python object must stay live.
TLS / X509 ctypes structures	Match source fields and native layout; do not hand-code arbitrary offsets.

Table A.3 FFI type mapping

Annex B: Source, Test and Example Inventory

Python API module	Public entries
qsc/abi010.py	7
qsc/asymmetric.py	54
qsc/buffers.py	2
qsc/encoding.py	16
qsc/entropy.py	14
qsc/errors.py	1
qsc/explicit_curve.py	5
qsc/explicit_kem.py	5
qsc/explicit_signature.py	5
qsc/explicit_slhdsa.py	5
qsc/hardware.py	2
qsc/hash.py	18
qsc/issuance.py	15
qsc/kdf.py	19
qsc/mac.py	37
qsc/memory.py	2
qsc/pqc.py	60
qsc/status.py	2
qsc/stream.py	10
qsc/symmetric.py	39
qsc/tls.py	87
qsc/version.py	9
qsc/x509.py	42
qsc/xof.py	6

Table B.1 Public-module inventory

Internal qsc/_native contains 329 registered function declarations in bindings.py and per-family _native modules for wrapper dispatch, pointer conversion and handle management. The tests directory contains 22 declared test modules and 148 distinct test methods, and examples/manifest.py describes 15 example groups. Their presence is not runtime evidence. The Visual Studio .pyproj and the cmd scripts are developer infrastructure; no pip packaging file exists in this uploaded archive.

Annex C: Implementation and Deployment Checklist

- Pin native QSCABI and QSC library revisions with verifiable DLL/SO/dylib hashes and target-specific installation paths.
- Check Python interpreter architecture against the native binary architecture and required operating system runtime.
- Run the ABI 0.10 compatibility check; in strict mode require every one of 329 registered public native functions.
- Represent native c_size_t, c_void_p, structs, enum selectors, argtypes and restype correctly and audit changes against actual headers.
- Use validated byte capacities and exact written lengths; reject failed status before reading any output.
- Use explicit context-manager/close release for TLS, X.509, stream and issuance handles; retain issuer/callback owners as needed.
- Keep secret data in mutable memory when feasible and wipe derived secrets, keys, seeds and native staging buffers.
- Prevent key/nonce reuse, tag bypass, unauthenticated plaintext release and unsupported cryptographic downgrade.
- Authenticate KEM peer identities and TLS/X.509 trust; verify expected hostnames, issuance role and revocation decisions.
- Treat optional hardware/explicit algorithms as feature-gated; validate hardware device presence independently.
- Run native-dependent QSCPY tests under --strict-abi --no-pause and retain per-platform logs and failure diagnostics.
- Exercise real TLS peers, certificate failure cases, PQC negative cases and physical hardware only where configured.
- Audit license notice, native dependency redistribution, platform signing, example credentials and package metadata.
- Separate source-only AST/static checks from actual compilation, runtime execution, native qualification or security certification.

The specification is an implementation reference and does not independently certify any cryptographic algorithm, device, protocol endpoint or binary distribution.